



(51) International Patent Classification:

G06N 3/00 (2006.01) G06N 3/08 (2006.01)
G06N 3/04 (2006.01) G11B 27/10 (2006.01)

(21) International Application Number:

PCT/GB2019/053198

(22) International Filing Date:

12 November 2019 (12.11.2019)

(25) Filing Language:

English

(26) Publication Language:

English

(30) Priority Data:

1818371.5 12 November 2018 (12.11.2018) GB
1910059.3 12 July 2019 (12.07.2019) GB

(71) Applicant: **RPPTV LIMITED** [GB/GB]: The Harrow Cottage, North Street, Midhurst, Midhurst Sussex GU29 9DJ (GB).

(72) Inventors: **FIELDING, Ben**; The Harrow Cottage, North Street, Midhurst Sussex GU29 9DJ (GB). **BUCHANAN, Will**; The Harrow Cottage, North Street, Midhurst Sussex GU29 9DJ (GB).

(74) Agent: **EIP**; Fairfax House, 15 Fulwood Place, London Greater London WC1V 6HU (GB).

(81) Designated States (unless otherwise indicated, for every kind of national protection available): AE, AG, AL, AM,

AO, AT, AU, AZ, BA, BB, BG, BH, BN, BR, BW, BY, BZ, CA, CH, CL, CN, CO, CR, CU, CZ, DE, DJ, DK, DM, DO, DZ, EC, EE, EG, ES, FI, GB, GD, GE, GH, GM, GT, HN, HR, HU, ID, IL, IN, IR, IS, JO, JP, KE, KG, KH, KN, KP, KR, KW, KZ, LA, LC, LK, LR, LS, LU, LY, MA, MD, ME, MG, MK, MN, MW, MX, MY, MZ, NA, NG, NI, NO, NZ, OM, PA, PE, PG, PH, PL, PT, QA, RO, RS, RU, RW, SA, SC, SD, SE, SG, SK, SL, SM, ST, SV, SY, TH, TJ, TM, TN, TR, TT, TZ, UA, UG, US, UZ, VC, VN, ZA, ZM, ZW.

(84) Designated States (unless otherwise indicated, for every kind of regional protection available): ARIPO (BW, GH, GM, KE, LR, LS, MW, MZ, NA, RW, SD, SL, ST, SZ, TZ, UG, ZM, ZW), Eurasian (AM, AZ, BY, KG, KZ, RU, TJ, TM), European (AL, AT, BE, BG, CH, CY, CZ, DE, DK, EE, ES, FI, FR, GB, GR, HR, HU, IE, IS, IT, LT, LU, LV, MC, MK, MT, NL, NO, PL, PT, RO, RS, SE, SI, SK, SM, TR), OAPI (BF, BJ, CF, CG, CI, CM, GA, GN, GQ, GW, KM, ML, MR, NE, SN, TD, TG).

Published:

— with international search report (Art. 21(3))

(54) Title: IMAGE CLASSIFICATION, GENERATION AND APPLICATION OF NEURAL NETWORKS

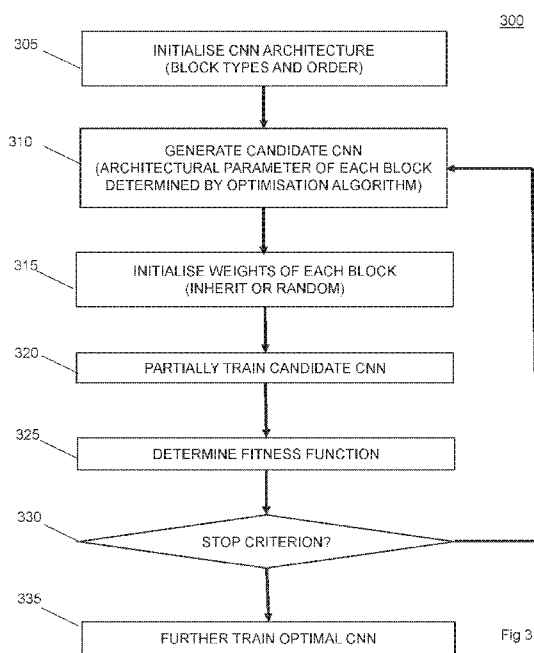


FIG. 3

(57) Abstract: The present invention relates to generating and training neural networks for application in numerous fields including object or event recognition in images including visual sequences. There is provided a computerised method of generating a neural network (NN), comprising generating successive candidate NNs (310, 545) using an optimisation algorithm (530), each NN having a number of connected blocks of layers (205), the layers having a plurality of neurons with connections having associated weights. Each block comprises fixed and variable architectural parameters (210X, 210Y), the or each variable architectural parameter being determined by an optimisation algorithm. Each candidate NN is trained using training data in order to update the weights of the candidate NN (320), and a fitness function score of the trained candidate NN is determined using validation data (330). If there is a block having the same architectural parameters from a previously trained candidate NN, the weights associated with layers of said block are inherited prior to training and fitness score determination (425).

Fig 3

IMAGE CLASSIFICATION, GENERATION AND APPLICATION OF NEURAL NETWORKS

Technical Field

The present invention relates to objection recognition in image processing. The present invention also relates to generating and training neural networks for application in numerous fields including object or event recognition in images including visual sequences.

Background

Image classification is useful for a wide range of applications. The use of machine learning in image classification allows for partial or total automation of applications requiring object recognition. However, in certain fields current machine learning technology is insufficiently flexible, is computationally expensive or too complex to deploy. This is particularly true in applications requiring recognition of a large number of different objects or scenes which may require a prohibitively large number of pre-trained recognition engines or retraining existing recognition engines for recognising new or modified objects or scene. The generation and training of each or a large number of machine learning based recognition engines for such applications may take significant time and/or require significant computer processing resources which may be commercially impractical and/or unaffordable. Whilst retraining or transfer learning techniques may be employed to reduce these constraints, the ability to experiment or create new model architectures is limited when using pre-trained models as the weights, and therefore general structure, of the model must be preserved in order to benefit from the latent information.

Convolutional neural networks (CNN) are a class of deep feed-forward artificial neural networks well suited to analysing visual imagery. However, designing and training CNN for new applications is very resource intensive, time consuming and expensive.

CNN comprise a number of layers of neurons connected within and between layers, the connections having associated weights which are adjusted during a training process so that the CNN is trained to respond to particular types of inputs such as images

of cats, or anomalies in medical images. Various types of layers are employed, for example convolving, pooling, non-linear functions, fully connected, normalisation layers. These may be connected in different combinations and with each layer having a wide range of possible parameters such as filter height, width and depth, number of filters, stride and padding, and non-linear function types. Because of the number of variables and their range of possible values, the number of possible CNN architectures is very large. It is therefore a difficult task to find an optimal CNN architecture for a new application.

Hand designed CNN are time consuming to design, requiring manual manipulation of layers, connections, and parameters, using trial and error and a deep understanding of the domain of application. Transfer learning takes advantage of the latent information of the existing model and is used to adapt existing CNN to the new applications, for example recognising cars and trucks instead of cats and dogs. However, transfer learning is not always successful as the weights of the original CNN may be finely tuned to its original data set and not able to generalise sufficiently well to other data sets, a problem known as overfitting.

There has been a large amount of recent interest in the task of designing architecture search strategies to replace this human-led trial and error process and provide an effective method to automatically design optimal architectures and associated hyper-parameters. Known automatic or computerised CNN design methods however are computer resource intensive, for example some require 20-250 GPU and hundreds of hours of processing time in order to generate an optimised and trained CNN for a new application. This makes them very expensive and therefore not generally available to smaller commercial users or research facilities.

Computerised CNN optimisation and training methods generate (often randomly) a number of models or candidate CNN having variable parameters – such as number and type of layers, number size and depth of filters, stride, padding, and other architectural parameters. Typically, these candidate CNNs are each partially trained using a sub-set of training data sets in order to adjust weights associated with connections between neurons in the layers. Fitness scores are calculated for each partially trained candidate CNN using validation data sets and a fitness function. Some of the parameters are then modified for each new candidate CNN using an optimisation

algorithm and ultimately an optimal CNN is determined based on the fitness scores. The optimal CNN is then fully retrained using a full set of training data

As more developments are made in progressing the internal components of CNNs, the task of assembling them effectively from core components becomes even more arduous. Many optimisation algorithms have been proposed for generating CNN for a new application.

Summary

According to a first aspect of the present invention, there is provided a computerised method of generating a neural network (NN), comprising generating successive candidate NNs using an optimisation algorithm, each NN having a number of connected blocks of layers, the layers having a plurality of neurons with connections having associated weights. Each block comprises fixed and variable architectural parameters, the or each variable architectural parameter being determined by an optimisation algorithm. Each candidate NN is trained using training data in order to update the weights of the candidate NN and a fitness function score of the trained candidate NN is determined using validation data. If there is a block having the same architectural parameters from a previously trained candidate NN, the weights associated with layers of said block prior are inherited by the current block prior to training and fitness score determination.

In an embodiment, a candidate NN is selected based on the fitness functions, and the selected NN is used to classify an image input to the selected NN.

In an embodiment, the blocks of layers may be arranged into a predetermined architecture with each block having a respective location within the predetermined architecture, and the weights are only inherited from blocks having the corresponding location within the predetermined architecture of a previously trained NN.

Each block may be of a predetermined block type having a number of predetermined layers and a number of variable layers dependent on the architectural parameter of the block.

The NN may be of any type, with the method being well suited to optimising convolutional neural networks (CNN).

The variable layers may comprise groups of layers, for example convolution, batchnorm, and ReLU for CNN. This means that the number of variable layers changes by a multiple of the group of layers.

The architectural parameter comprises one or more of the following: number of layers; filter sizes; filter depths; number of filters; filter strides; filter paddings; filter biases; filter dilations; connections between layers; type of convolution.

In an embodiment the number of variable layers is used as the architectural parameter which varies in the NN generating method, however other architectural parameters may alternatively be used. However, by only using one or a sub-set, the computing resources required is significantly reduced compared with known methods.

The weights may be inherited according to a non-linear function from a corresponding block of the last or best previous candidate NN, the best previous NN determined by its fitness function score.

The optimisation algorithm is a particle swarm optimisation (PSO) algorithm, each particle corresponding to the architectural parameter of each block in the candidate NN. The PSO may comprise acceleration coefficients which are adapted over the duration of a search according to a non-linear function.

In an embodiment an optimal NN is determined dependent on the fitness function scores of the candidate NN, retains the weights from the training step and is then further trained using the training data and/or validation data. This approach can also significantly reduce the computing resources required to generate an optimal and fully trained NN.

In an embodiment, a number of local best candidate NN are selected using the fitness function scores, and these are used as an ensemble of NN to classify an image input to the ensemble. The classification of the ensemble may be determined based on the classifications from NN of the ensemble, for example using majority or plurality voting.

NN generated according to the method may be used in any suitable application. Examples include classifying an image as belonging to one class such as “dog” and may include recognising one or more of the following: an object or type of scene in an image; an anomaly in a medical image; an event in a visual sequence of images; a person’s face or gait; security or medical applications; media production applications

such as recognising events in a video for the purpose of adding appropriate sound effects such as footsteps, a door closing, a punch impacting an actors face, a dropped vase hitting a floor, a lion roaring, a car spinning, an aircraft taking off, an actor talking and many other possibilities.

A system employing an NN generated according to one these methods may be arranged to automatically initiate an action in response to recognising a particular event, for example: changing the mode of a security system; sending a medical alert; generating an onscreen menu of options for changing audio data associated with the recognised event; changing audio data associated with the recognised event or scene – for example changing the background music or soundtrack associated with a sequence of frames in a media post-production application from a pub sound file to a desolate forest sound file when the corresponding scenes are recognised.

According to another aspect of the present invention, there is provided an apparatus for generating a neural network (NN), comprising a memory and a processor which when executing instructions stored on the memory is arranged to perform a method of generating a neural network (NN). The method comprises generating successive candidate NNs using an optimisation algorithm, each NN having a number of connected blocks of layers, the layers having a plurality of neurons with connections having associated weights. Each block comprises fixed and variable architectural parameters, the or each variable architectural parameter being determined by an optimisation algorithm. Each candidate NN is trained using training data in order to update the weights of the candidate NN and a fitness function score of the trained candidate NN is determined using validation data. If there is a block having the same architectural parameters from a previously trained candidate NN, the weights associated with layers of said block prior are inherited by the current block prior to training and fitness score determination.

According to yet another aspect of the present invention, there is provided a neural network (NN) generated according to a method which comprises generating successive candidate NNs using an optimisation algorithm, each NN having a number of connected blocks of layers, the layers having a plurality of neurons with connections having associated weights. Each block comprises fixed and variable architectural parameters, the or each variable architectural parameter being determined by an

optimisation algorithm. Each candidate NN is trained using training data in order to update the weights of the candidate NN and a fitness function score of the trained candidate NN is determined using validation data. If there is a block having the same architectural parameters from a previously trained candidate NN, the weights associated with layers of said block prior are inherited by the current block prior to training and fitness score determination.

In another aspect, there is provided a method of generating an ensemble of NN for classifying images. The individual NN may be generated by any of the described methods herein, or a different method. In an embodiment the additional NN are determined from previous candidate NN using the configuration corresponding to their best (best) local fitness score. In an alternative embodiment, the additional NN are generated using the best (fitness score) two (or more) blocks for each block position in the architecture from the candidate NN and generating an ensemble of additional NN by combining different combinations of the determined blocks. The ensemble may then be used to classify the image.

In another aspect there is provided a media post-production apparatus for processing a sequence of images, the apparatus comprising:

- a processor and memory to implement a plurality of NN each capable of a different classification of an image;
- a classification engine to use the NN to classify one or more of the images;
- a sound engine to associate a sound with the images depending on their classification.

The media post-production apparatus may use NN generated according to the methods described herein.

The sound may be a background sound file. The classification engine may determine a sequence of images having the same classification, and the images may have respective timecodes in a video file.

The sound engine may associate a portion of the sound file having a duration corresponding to the duration between the timecodes of the first and last image in the sequence.

According to another aspect, there is provided a method of classifying an image including recognising an object or scene. The computerised method comprises generating successive candidate NNs using an optimisation algorithm, each NN having a number of connected blocks of layers, the layers having a plurality of neurons with connections having associated weights. Each block comprises fixed and variable architectural parameters, the or each variable architectural parameter being determined by an optimisation algorithm. Each candidate NN is trained using training data in order to update the weights of the candidate NN and a fitness function score of the trained candidate NN is determined using validation data. If there is a block having the same architectural parameters from a previously trained candidate NN, the weights associated with layers of said block prior are inherited by the current block prior to training and fitness score determination. The candidate having the best fitness score is selected to recognise the object in the image.

Further features and advantages of the invention will become apparent from the following description of preferred embodiments of the invention, given by way of example only, which is made with reference to the accompanying drawings.

Brief Description of the Drawings

Figure 1 is a representation of a convolutional neural network (CNN);

Figure 2 shows a schematic of a predetermined architecture of a CNN according to an embodiment;

Figure 3 shows a method of generating a CNN according to an embodiment;

Figure 4 shows a method of inheriting weights according to an embodiment;
Figure 5 is a schematic of an apparatus according to an embodiment;
Figure 6 represents a particle optimisation algorithm according to an embodiment;

Figure 7 illustrates different functions for inheriting weights over the course of an optimisation;

Figure 8 illustrates different functions for adapting acceleration coefficients over the course of an optimisation;

Figure 9 illustrates particle positions over the course of an optimisation;

Figure 10 illustrates particle positions over the course of an optimisation;

Figure 11 illustrates confusion plots for an example test of an optimal CNN generated according to an embodiment;

Figure 12 shows error rates for candidate CNN having different numbers of variable layers in different block locations;

Figure 13 shows a method of generating a CNN ensemble according to an embodiment;

Figure 14 illustrates a user interface for a media post-production system according to an embodiment;

Figure 15 shows a method of operating a media post-production system according to an embodiment;

Figure 16 illustrates a media post-production system according to an embodiment;

Figure 17 shows a method of generating a CNN ensemble according to another embodiment;

Figure 18 illustrates a projection of candidate particles used for a local best CNN ensemble embodiment; and

Figure 19 illustrates a projection of candidate particles used for a look-up CNN ensemble embodiment.

Detailed Description

Figure 1 is a schematic representation of a simple convolutional neural network (CNN) 100 which comprises a plurality of layers 105A-D each having a plurality of

neurons 115 with connections 110 to other neurons within and between the layers. The connections 110 each have an associated weight and each neuron 115 input signals from the incoming connections according to their respective weights and processes these according to a predetermined function in order to generate an output which forms an input for one or more other neurons. A more detailed representation of a layer 105X is shown which includes multiple neurons in three dimensions – height h, width w, and depth d. In addition to these dimensions, each layer will have a number of additional hyperparameters such as stride, padding, dilation, type of convolution (eg depthwise or spatial) as will be known to those skilled in the art. As is also known, the CNN is trained on training data which is used to adjust the weights associated with different connections so that the CNN is trained to recognise certain patterns, such as a particular type of image.

Modern CNN typically comprise tens or even hundreds of layers, which include a series of specialised layers including convolution, non-linear activation function (eg ReLU), batch-norm (normalise the inputs to nonlinearities), pooling or downsampling and if used for classification tasks a fully connected layer to classify an input. The particular arrangement of the layers and their respective hyperparameters, known herein collectively as architectural parameters, represent the architecture of a particular CNN and this can be optimised for each particular application. The process of optimising requires at least partially training a candidate CNN using example inputs corresponding to the application and testing its ability to correctly classify other examples of inputs corresponding to the application. One or more of the architectural parameters is adjusted and the process repeated in order to find an optimal architecture for the particular application. The CNN with the optimal architecture is then reinitialised and fully trained. The search space of possible architectures can be enormous, and as each requires at least partial training, the computational resources required to find and train an optimal architecture are very large.

Figure 2 illustrates a predetermined architecture template according to an embodiment. The predetermined or skeleton architecture 200 includes a number of connected blocks of layers 205A-E, each block comprises a number of predetermined layers 210X and a number of variable layers 210Y. The blocks may be of different types each type having a different arrangement of predetermined layers 210X. For example,

block 205B comprises a convolutional layer followed by a BatchNorm layer followed by a ReLU layer which is then followed by a variable number of variable layers 210Y. Blocks 205A, C and D are of the same block type all having the same predetermined layers 210X, whereas block 205E is of a different type having a different arrangement of predetermined layers 210Z. In this embodiment the predetermined architecture has some intermediate layers 215 between the blocks of layers, including downsampling or pooling layers. Each block 205A-E has a respective position or location within the predetermined architecture, for example block 205C being the third block coupled between blocks 205B and 205D.

The variable layers may be arranged into groups of layers such as Convolution, BatchNorm, ReLU, so that the number of variable layers is a multiple of groups of these three layers.

The predetermined architecture has a number of architectural parameters including the width height and depth of each layer, the layer configuration (eg convolution or pooling), the stride and padding of each layer, the size of the kernels (filters) for each layer, the number of variable layers 210Y. The predetermined architecture may be based on an existing CNN which is optimised for an application of interest, or one related to this. One or more architectural parameters of the predetermined architecture are then iterated according to an optimisation algorithm in order to determine an optimal CNN for the application. However only a sub-set of the architectural parameters are varied with each new CNN in order to reduce the search space for an optimal CNN, and therefore to reduce the computing resources required. In this embodiment, the number of variable layers 210Y is the only architectural parameter varied, however in alternative arrangements different and/or additional architectural parameters can be used.

An example CNN architecture suitable for the predetermined is the VGG-16 architecture described in K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," CoRR, vol. abs/1409.1556, 2014. This includes intermediate down-sampling layers 215 which reduce the width and height of the network whilst simultaneously increasing the number of features maps as the network progresses. The decrease in spatial size can also be seen as a gradual increase

The candidate CNN is then trained using training data at step 320. The training data comprises a series of known inputs which are processed through the candidate CNN to generate an output which is compared with a wanted output. The difference or error is used to update weights of the connections as would be appreciated by those skilled in the art. In this embodiment the training process is backpropagation of errors with stochastic gradient descent although alternative CNN training methods could be used. Each CNN is trained for a limited number of epochs using the training data set. The amount of training is significantly less than other known methods of determining an optimal CNN because of weight sharing or inheritance which will be described below. This can significantly limit the number of training epochs (or training data) required which significantly reduces the computing resources required.

A fitness function score of the trained candidate CNN is then determined at step 325. This uses validation data which is input to the trained candidate CNN and the output is assessed against an expected output using a fitness function. Each trained candidate CNN will be stored in the computing apparatus together with its fitness function score.

The method 300 then determines whether a stop criterion has been met at step 330. The stop criterion may be a set number of iterations, a fitness function score exceeding a threshold, or some other criterion. If the stop criterion has not been met, the method returns to step 310 to generate a new candidate CNN for training and testing.

If the stop criterion is met, an optimal CNN is determined from the candidate CNN based on their fitness function scores at step 335. Typically, the candidate CNN having the best fitness function score (eg lowest error or maximum accuracy) is assigned as the optimal CNN. For the fitness function of the embodiment described below, the lowest fitness score is selected.

The optimal CNN is then further trained at step 340. The optimal CNN retains the weights determined from the initial training step 320 as its initial weights and is then fine-tuned by further trained on the training and validation data combined for a limited number of epochs. By retaining the weights from the optimisation method, the overall training process is faster and requires reduced computing resources compared with reinitialising the weights randomly before fully training the optimal CNN as used

in other methods. The fully trained CNN may then be tested using additional test data (not shown).

Figure 4 illustrates a method of initialising weights for step 315. The method 400 comprises for each block of the candidate CNN, whether there is a previous CNN candidate (from an earlier iteration of the method) with a block having the location within the architecture and the same architectural parameter as the current candidate CNN – step 405. If not, the method allocates random weights to the block at step 410, and then determines whether there are further blocks to consider at step 415. If there are the method returns to step 405, if there are no more blocks, the method returns to step 315 of Figure 3 (420).

If there is an earlier candidate CNN having a block with the same architectural parameter as the current block of the current candidate CNN, and the same block location, then the weights from the earlier block are inherited by the current block – step 425. In other words, the weights following training (step 320) of connections of the block of the previous candidate CNN are applied as the initial weights of the corresponding connections of the current block. The method then moves to step 420.

This method 300 and 400 significantly reduces the computing resources required compared with other optimisation methods. The reduction is in part due to the reduced number of training examples that must be used for each candidate architecture when evaluating against the fitness function. Without any parameter sharing, the only way to get a representative view of the performance of the candidate architecture is to train it for a long time and then validate. In this way the computational cost of each fitness function evaluation is drastically reduced by considering it to be an ongoing process, rather than a standalone, repeated process. This means that the fitness scores will tend to improve throughout the optimisation process, even for architectures that are exactly the same as they were in a previous iteration. This is contrary to other methods that don't use weight sharing, where a fitness evaluation could be considered to stand on its own, regardless of when in the optimisation process it was performed. In the present embodiment the fitness function evaluations won't be good representatives of eventual performance initially, but they will improve drastically between iterations, with these early iterations effectively exploring candidate CNN with very different architectural parameters. Later iterations will be significantly improved as the candidate CNN will

mostly be inheriting existing good weights. At this point each evaluation is more representative of eventual performance and is more fine tuning of the eventual optimised architecture.

The inheriting step 425 in this embodiment uses a non-linear function to determine which weights to inherit where there is more than one corresponding block from earlier iterations. The weights are inherited from either the last candidate CNN to have a corresponding block, or the previous candidate CNN having a corresponding block with the best fitness function. The non-linear function may be a cosine function, although another non-linear function may be used. Similarly, the source of weights to be inherited may be different, for example the first and second best candidate CNN having a corresponding block.

Fully evaluating each candidate CNN would require prohibitive amounts of time and resources as each architecture must be trained and validated to obtain a fitness score. Early evaluation can be employed by using a smaller training set and/or a reduced number of epochs before determining the fitness function score using the validation data. Often using early evaluation does not provide a realistic view of the performance of a candidate CNN as an architecture may train very successfully initially but later plateau before reaching an acceptable level of accuracy. This problem is addressed using the above weight inheriting arrangement so that the initial weights lead to improved training and fitness function scores with a more realistic view of the performance of the candidate CNN.

This weight inheriting method effectively means that the candidate CNN are continually trained during the optimisation process leading to significantly less training overall. In other words, less training epochs and/or fewer training examples are required as each candidate CNN does not need to be trained from scratch to determine a fitness score, but rather the training is effectively shared amongst the CNN candidates leading to less training overall and therefore reduced computing resource requirements compared with other methods, with the fitness scores becoming more accurate as optimisation/training progresses. This allows the search space of candidate CNN to be traversed more quickly

In an embodiment the population of continually evolving candidate CNN architectures are jointly trained by maintaining a lookup table of convolutional filter

parameters and fully connected layer weights. The lookup table consists of a simple key-value store, where the key takes the form of a string concatenation of the integer block number in the architecture, with the integer size of the block (i.e. the number of layers in the block minus 1; zero indicates a single layer) separated by a period. A key thus takes the form $\alpha.(w-1)$ where α is the specific number of the block in the skeleton architecture, and w is the number of layers in the block. Each value in the key-value store is itself a smaller key-value store, consisting of two key-value pairs, i.e. the best performing parameters, and the last used parameters, for each distinct block & size. This allows for checking if a specific block has been constructed to a certain size before, and to inherit the weights of that block, thereby gradually training blocks as the architecture search space is explored. Using best and last CNN can be an improvement over only using the best weights as this has the potential downside of limiting the exploration of the search space since it ensures that any training run that does not increase performance by the end of the run will be discarded, in favour of the original parameters. Such an approach can be limited in its exploration capability.

Different functions can be used to select between the best and last weights such as those shown in figure 7, which shows the likelihood $B = 0$ to 1 of selecting the best weight over a number of fitness function evaluations. Selecting only the best weights can lead to a cycle of limited exploration, whereby the candidate CNN become repeatedly stuck retraining parameters but never achieving a better validation score, and therefore discarding its progress. Providing a fixed 50:50 chance of inheritance from best or last allows for exploration but promoting superior results with an even chance; though this could be superior to the best weight only approach. Using the annealing schedule below for B allows the chance of inheriting from best, rather than last, to gradually increase from 0% to 100%.

$$\beta = 1 + \frac{\cos\left(\pi\left(1 - \frac{\emptyset}{\Phi}\right)\right)}{2}$$

where $\emptyset$ and Φ represent the current and total number of fitness function evaluations respectively. Φ can be calculated by $m+mxT$, where m represents the swarm population and T represents the number of iterations for the optimisation algorithm. When storing the weights for a specific block in the architecture, the weight tensors are transferred

into system RAM in order to save the on-board GPU memory for larger batch sizes and larger potential network architectures.

It will be appreciated that different predetermined architectures and/or architectural parameters could be used in alternative embodiments. It will also be appreciated that whilst the embodiments have been described with respect to CNN, other types of artificial neural networks (ANN) could also be optimised and trained using these methods and apparatus, for example other types of feedforward or recurrent neural networks (RNN).

Figure 5 is a schematic of an apparatus for generating a CNN according to an embodiment. The apparatus 500 comprises a processor 505 and a memory 510 which can together implement the methods 300 and 400. This figure illustrates the data-structures and procedural flows involved in implementing these methods. The memory 510 comprises data structures for one or more predetermined architectures 520, a layer library 525, and optimisation algorithm 530, training data 535, validation data 540, and a weight inheriting algorithm 560. The optimisation algorithm (530) allocates one or more layers from the layer library (525) to the predetermined architecture (520) in order to generate candidate CNN (CNN1-n) which are each temporarily stored prior to (partial) training using a subset of the training data (535). The weight inheriting algorithm (560) may adjust the initial weights of the candidate CNN prior to training using a corresponding block from a previous candidate CNN, if any. The partially trained CNN (Train CNN1-n) are stored for subsequent use. The trained candidate CNN are also validated by applying validation data (540) in order to determine respective fitness function score (555) which are also stored for subsequent use. When an optimal CNN is determined 565, this retains its existing weights and is then fully trained using the training data 535 and validation data 540 in order to generate the optimised and fully trained CNN with reduced computational resources compared with known methods. Fine tuning using training data 535 and validation data 540 combined alleviates the overfitting on training data 535 alone from the previous training and allow the optimal CNN to generalise to different data.

The training and validation data 535 and 540 correspond to specific applications for which the generated CNN is optimised and trained. For example, the CNN may be optimised and trained to recognise certain anomalies in medical images, or to recognise

a certain person's face. CNN generated using the embodiment may also be used to recognise or identify patterns in a visual sequence such as a person by their gait, which may be used for security purposes or medical purposes for example if the normal gait is different indicating a stroke. Other visual sequence pattern recognition applications could include a sporting event such as a goal in a soccer match or footsteps in a movie segment. The CNN generation method of the embodiments may be used in media production applications, for example to optimise and train CNN to recognise events in a video sequence – some examples include footsteps, door closing, a person speaking, glass smashing, a car accelerating, and so on.

A method or apparatus in accordance with an embodiment may be arranged to automate and act in response to detecting a predetermined visual sequence pattern. For example, recognition of a normal gait of a person approaching their house may automatically disable security, or recognition of an abnormal gait of the person within the house may automatically trigger a medical alert. In media video production equipment, recognition of footsteps in a movie sequence may trigger replacement of the exiting footstep audio with enhanced footstep audio, or a menu screen offering various options related to enhancing the associated audio. Gait recognition may be used for tracking actors for dialogue placement.

An optimisation algorithm according to an embodiment is represented in Figure 6. The algorithm uses particle swarm optimisation (PSO) where each particle represents the variable architectural parameters of each block of a candidate CNN. The search space of possible particles is represented by the 3D axes, and particles representing candidate CNN having different architectural parameters are generated by the optimisation algorithm based on previous CNN candidates and their respective fitness function scores as described below.

Particle Swarm Optimisation (PSO) is a stochastic optimisation technique that relies on a population X of m individuals, each with a specific position in the search space defined by a fixed-length vector R_n . This is described in more detail in R. Eberhart and J. Kennedy, "A new optimizer using particle swarm theory," in *Micro Machine and Human Science*, 1995. MHS'95., Proceedings of the Sixth International Symposium on. IEEE, 1995, pp. 39–43.

Each position in the search space represents a distinct set of parameters to an objective function f – in the embodiments the fitness function. The fitness of an individual particle represents the result of evaluating the objective function f with the position of the particle as parameters. The goal of PSO is to minimise or maximise the objective evaluation by finding the best overall particle position $\text{argmin } f(x)$ or $\text{argmax } f(x)$. The individual particles in the population are initialised with a random position in the search space, usually by drawing their values from a uniform distribution U , bounded by defined upper (bu) and lower (bl) bounds. The particles are then iteratively evaluated and conduct the search process by following personal and global best solutions in order to attain global optimality. Specifically, as the particles are moved around the search space, the best positions found so far, along with their fitness scores, are stored for each individual particle. These are referred to as the ‘local best’ solutions. The best solution of the overall swarm is referred to as the ‘global best’ solution and indicates the best set of parameters that the algorithm has as-yet found for the objective function. (1) & (2) denote the velocity and position updating operations for each particle respectively.

$$V_i^t = \omega V_i^{t-1} + c_1 r_1 (P_i - X_i^{t-1}) + c_2 r_2 (P_g - X_i^{t-1}) \quad (1)$$

$$X_i^t = X_i^{t-1} + V_i^t \quad (2)$$

where c_1 and c_2 denote acceleration coefficients, and r_1 and r_2 are random vectors drawn from $U(0, 1)$ to introduce stochasticity. P_i and P_g represent the personal and global best solutions respectively, with ω was the inertia weight. X_i^{t-1} and V_i^{t-1} represent the position and velocity of the particle i from the previous ($t - 1$) iteration, respectively. The process is repeated over a defined number of iterations, or until a stop criterion is met. The velocities of the particles are updated by using three components, i.e. the existing velocity, the distance between the current position and the best position of this particle so far (local best), and the distance between the current position and the swarm leader (global best). Each of the three main components thus described are weighted to control the effect they have on the resulting velocity and position updates. These search weights take the form of w , c_1 , and c_2 , where w controls the impact of the previous velocity, c_1 controls the effect of the local best, and c_2 controls the effect of the global

best. The standard PSO model employs pre-determined, fixed search weights, thereby defining the magnitude of the effect of the local and global bests on the resultant velocity and defining the overall magnitude of the velocity itself.

In order to improve the performance of the standard PSO algorithm, a modified version according to an embodiment can be employed.

A method for mapping a vector of integers to a full convolutional architecture by ‘stacking’ layers in each block is defined according to the value in the specific index of the architecture vector. In this way, particles can move through an n dimensional space, where n represents the number of tuneable blocks in the architecture. The task of generating the architecture of a model (candidate CNN) as a minimisation of an objective function $f(x)$ (defined in Fig.4), where x represents an abstraction of network architecture into a single point in a navigable multidimensional search space and $f(x)$ represents the error rate of the model when evaluated on the validation set. This involves discovering the optimal value of x which produces the minimal error rate when evaluated using the fitness function, as shown by (3).

$$\underset{x}{\operatorname{argmin}} f(x) = \{x \mid x \in S \wedge \forall y \in S : f(y) \geq f(x)\} \quad (3)$$

where,

$$S \subset \mathbb{N}^n \mid \forall x \in S : +0 \leq \{x_0, \dots, x_n\} < 10 \quad (4)$$

But the search space can be explored using,

$$S \subset \mathbb{R}^n \mid \forall x \in S : 0 \leq \{x_0, \dots, x_n\} < 10 \quad (5)$$

and rely on the implicit integer-cast to function as a form of regularisation by only allowing large, or multiple small, movements to modify the structure. This can be achieved by min-max scaling the position values into a desired range and then converting into integers representing the number of layers to add to each block. This can be simply performed by multiplying each position value by the upper bound of the range and use the values 0 and 1 as the lower and upper bounds for optimisation respectively. In order to optimise the objective function, the following enhanced particle swarm optimisation can be used

Each individual architecture solution or candidate CNN in the search space is considered as a position in an n-dimensional space where n represents the number of

distinct blocks in a skeleton or predetermined architecture. Instead of using fixed acceleration coefficients as in the original PSO model, adaptive search parameters based on non-linear functions can be employed. These may include: 1) cosine functions with an equal crossover in the centre, 2) cosine functions with a later crossover, and 3) cosine functions with no crossover. These strategies can be seen in figure 8.

A population of individual particles is initialised as random positions in the search space, where each dimension in each particle is drawn from a uniform distribution:

$$X = U(b_l, b_u) \quad (6)$$

where b_l and b_u are the lower and upper boundaries of the search space, respectively. The velocity of each of the particles is initialised:

$$V_i = U(v_{min}, v_{max}) \quad (7)$$

where,

$$\begin{aligned} v_{max} &= \lambda(b_u - b_l) \\ v_{min} &= -v_{max} \end{aligned} \quad (8)$$

and λ is 0.2.

Once the swarm has been initialised, the optimisation process can begin. It starts with updating the inertia weight and both acceleration coefficients according to the specific strategies chosen. Next, each particle X_i is processed with the following steps. First the velocity of the particle is updated using the search weights and the distances between the current position and the local and global best positions as defined in (1). Using the velocity, the new position of the particle is calculated based on the previous position as illustrated in (2). The fitness of the particle is evaluated using the objective or fitness function provided. The fitness score of X_i is compared against those of the previous personal best position P_i and the global best solution, respectively. The local best position is then updated according to (9).

$$P_i = \begin{cases} X_i, & \text{if } f(X_i) < f(P_i) \\ P_i, & \text{otherwise} \end{cases} \quad (9)$$

Whilst the global best is similarly updated according to (10).

$$G = \begin{cases} \argmin f(P_i), & \text{if } \min < f(G) \\ P_i, & \\ G, & \text{otherwise} \end{cases} \quad (10)$$

In practice, in order to avoid unnecessary overhead, the fitness scores are stored for each evaluation for the score comparisons, rather than re-calculating the fitness score for each updated particle position. This process then repeats over all particles, and all iterations, until a certain stop criterion has been met, i.e. the maximum number of iterations. Once the iterations have completed, the final output of the system is the global best position G and its fitness value $f(G)$, representing the best arguments to minimise the objective function ($\text{argmin } f(x)$) and the fitness value respectively.

Using adaptive search parameters in the PSO enables the search to favour local exploitation in early iterations and global exploration in final iterations. This avoids prematurely optimising to a local minimum before the possible architectures have been trained for a reasonable number of iterations. Otherwise it is likely that the large improvements in error rate that can be seen with the first few training iterations would result in a rapid clustering of all of the particles into one area after following the global best solution.

Combined with the continual training embodiment described above, initially large gains can be expected no matter where a particle moves, owing to the initial training of the networks up to a reasonable level of performance. Because of this, it is desirable that each particle be allowed to explore its own space initially, rather than move towards the global best. This ensures that the particles perform useful exploration in these initial stages, by moving towards the area with the greatest improvements around themselves. This can be achieved by setting the ratio between the local search weight and the global search weight to a high value initially. However, later in the training process, the performance gains from each iteration will slow down significantly, as the networks come closer to achieving their optimal performance. At this point, it is desirable to obtain the best performing, single network from the population of individuals. This allows the particles' positions to trend towards the position of the best performing particle, to explore together around the position and achieve even better performance. At this later iteration stage, the previously described ratio between local and global search weights should be reversed, promoting a high ratio of global to local.

The optimisation algorithm of the embodiment achieves this using adaptive acceleration coefficients, where the search weights can change depending on the current

iteration number in a non-linear manner. Figure 8 shows the search weight of the acceleration coefficients c_1 and c_2 both changing according to inverse cosine functions, with the local weight c_1 being dominant during initial iterations and the global weight c_2 dominating in later iterations. Using a non-linear function maintains the dominance of the initial local searching of each particle for longer so that the particles can explore their local space more efficiently without prematurely heading towards the global best during the initial training stages. In other words, this approach prevents the particles from quickly abandoning their local search space in favour of pursuing the best position, and thereby all becoming stuck in the same local minima. Cosine based non-linear functions for use in an embodiment are shown below:

$$\begin{aligned} c_1 &= q + \frac{Q-q}{2} \cos\left(\pi \left(1 - \frac{t}{T}\right)\right) + 1 \\ c_2 &= q + \frac{Q-q}{2} \cos\left(\pi \frac{t}{T}\right) + 1 \end{aligned} \quad (11)$$

where t refers to the current iteration number, T refers to the total number of iterations to be performed for the optimisation run, q refers to the lower bound for the search weight, and Q refers to the upper bound for the search weight. The Cosine Equal Crossover variant (Fig.8a) is created using $q = 0.5$, $Q = 2.5$ for both c_1 and c_2 . The Cosine Late Crossover variant (Fig.8b) is created using $q = 0.5$, $Q = 2.5$ for c_1 and $q = 0.5$, $Q = 2.5$ for c_2 . The Cosine No Crossover variant (Fig.8c) is created using $q = 2.0$, $Q = 2.5$ for c_1 and $q = 0.5$, $Q = 2.5$ for c_2 . Figure 9 illustrates an example evolution of the optimisation algorithm over the search space.

Fig.10 demonstrates the effects of the late crossover cosine search weight strategy on the local bests of each particle and the overall global best. The x and y axes represent the particle position projected into two-dimensional space using Principal Component Analysis (PCA). The z axis represents the fitness value for the particle after being evaluated on the validation set. Fig.10a shows how initially the particles explore their own space, improving their fitness scores but not converging on a single location, as well as how they begin to converge on the x and y axes later as the search weights begin to favour following the global best. This can also be seen in the positions of the local and global bests in Fig.10b and Fig.10c, which eventually converge around a single point after gradually narrowing focus and improving fitness scores.

The embodiments may be used with any suitable training method, for example backpropagation and gradient-based optimisation, for example stochastic gradient descent.

The following example function pseudocodes can be implemented in embodiments.

1) A PSO function with adaptive acceleration coefficients

```

1.   Function PSO ( $\chi$ ):
2.    $f \leftarrow \text{ObjectiveFunction}()$ 
3.    $m \leftarrow \text{population}$ 
4.    $n \leftarrow \text{dimensions}$ 
5.    $b_l \leftarrow 0$ 
6.    $b_u \leftarrow 1$ 
7.    $x_{m,n} \leftarrow U(b_l, b_u)$ 
8.    $P \leftarrow X$ 
9.    $G \leftarrow X_{\text{argmin}(f(X))}$ 
10.   $V_{\max} \leftarrow \lambda(b_u - b_l)$ 
11.   $V_{\min} \leftarrow -V_{\max}$ 
12.   $V \leftarrow U(V_{\min}, V_{\max})$ 
13.   $mode \leftarrow [\text{'fixed'} | \text{'linear'} | \text{'cosine'} | \text{'AGPS01'}]$ 
14.  for  $t \leftarrow 0, \dots, T$  do
15.     $lr \leftarrow \text{SetLearningRate}(lr, t)$ 
16.     $w, c_1, c_2 \leftarrow \text{SetSearchWeights}(T, t, mode)$ 
17.    for  $i \leftarrow 0, \dots, m$  do
18.       $V_i \leftarrow w V_i + c_1 r_1 (P_i - X_i) + c_2 r_2 (G - X_i)$ 
19.       $X_i \leftarrow X_i + V_i$ 
20.      if  $f(X_i) < f(P_i)$  then
21.         $P_i \leftarrow X_i$ 
22.      if  $f(P_i) < f(G)$  then
23.         $G \leftarrow P_i$ 

```

2) Inherited weights function

```

1.   Function LOOKUP ( $\beta$ )
2.   If  $\beta > U(0,1)$  then
3.     return best
4.   else
5.     return last

```

3) PSO adaptive acceleration coefficients

```

1.  function SETSEARCHWEIGHTS ( $T, t, mode$ )
2.    if mode is fixed then
3.       $w \leftarrow 0.6$ 
4.       $c_1 \leftarrow 0.9$ 
5.       $c_2 \leftarrow 0.9$ 
6.    else if mode is linear then
7.       $q \leftarrow 0.5$ 
8.       $Q \leftarrow 2.5$ 
9.       $w \leftarrow 0.6$ 
10.      $c_1 \leftarrow Q - \frac{t}{T}(Q - q)$ 
11.      $c_2 \leftarrow q + \frac{t}{T}(Q - q)$ 
12.    else if mode is cosine_equal_cross then
13.       $q \leftarrow 0.5$ 
14.       $Q \leftarrow 2.5$ 
15.       $w \leftarrow 0.6$ 
16.       $c_1 \leftarrow q + \frac{Q-q}{2} \cos\left(\pi \left(1 - \frac{t}{T}\right)\right) + 1$ 
17.       $c_2 \leftarrow q + \frac{Q-q}{2} \cos\left(\pi \frac{t}{T}\right) + 1$ 
18.    else if mode is cosine_late_cross then
19.       $q \leftarrow 1.5$ 
20.       $Q \leftarrow 2.5$ 
21.       $w \leftarrow 0.6$ 
22.       $c_1 \leftarrow q + \frac{Q-q}{2} \cos\left(\pi \left(1 - \frac{t}{T}\right)\right) + 1$ 
23.       $m \leftarrow 0.5$ 
24.       $n \leftarrow 2.5$ 
25.       $c_2 \leftarrow q + \frac{Q-q}{2} \cos\left(\pi \frac{t}{T}\right) + 1$ 
26.    else if mode is cosine_no_cross then
27.       $q \leftarrow 2.0$ 
28.       $Q \leftarrow 2.5$ 
29.       $w \leftarrow 0.6$ 
30.       $c_1 \leftarrow q + \frac{Q-q}{2} \cos\left(\pi \left(1 - \frac{t}{T}\right)\right) + 1$ 
31.       $m \leftarrow 0.5$ 
32.       $n \leftarrow 2.5$ 
33.       $c_2 \leftarrow q + \frac{Q-q}{2} \cos\left(\pi \frac{t}{T}\right) + 1$ 
34.    else if mode is AGPS01 then
35.       $w \leftarrow 0.6$ 
36.       $c_1 \leftarrow \frac{t}{T} + 1.25$ 
37.       $c_2 \leftarrow \frac{-2.05}{T} + 2.55$ 
38.  return,  $w, c_1, c_2$ 

```


4) A fitness or objective function

1. **function** OBJECTIVEFUNCTION(position)
2. $model \leftarrow \text{InitModel}(\text{position})$ *Initialise the model using the particle position vector*
3. **if** weight_sharing **then**
4. $model.weights \leftarrow \text{BlockLookup}(\text{position})$ *Construct the weight lookup keys from the model structure*
5. *Determine the weight lookup strategy*
6. *Iteratively check the weight lookup table for existing weights*
7. *Load the existing weights onto the model*
8. Train(model) *Train the model for a single epoch (or a defined number of batches)*
9. $error_rate \leftarrow \text{Validate}(\text{model})$ *Evaluate the model on the validation set to determine the error rate*
10. **if** weight_sharing **then**
11. BlockStore(position,model,weights,error_rate) *Iteratively update the weights in the weight lookup table*
12. **return** (error_rate/100) *Return the error rate as the fitness for this function evaluation*

Compared with some embodiments, the known method for evaluating architecture optimisation systems is to separate out the optimisation process from the final model training process. Contrary to this approach, some embodiments integrate the optimisation process into the training process for the final or optimal CNN to be tested. Once the optimisation process has completed, the best performing candidate CNN is then fine-tuned on a combined training set consisting of the training and validation data together for a small number of epochs. In this way the training of the final network is embedded in the optimisation process, meaning the architecture design and training are performed as one task. By using this approach, the architecture design and training processes are coupled together and remove some of the high barrier-to-entry for building CNNs for new problems.

Experimentation performed on an embodiment using the CIFAR-10 and CIFAR-100 datasets for standardised image classification training shows its effectiveness. CIFAR-10: The CIFAR-10 dataset consists of 60,000 images equally split over 10 classes (6,000 per class). The dataset divides into 50,000 training images and 10,000 test images. These training images are further divided into 45,000 training and 5,000 validation, whereby the validation set is used to generate the fitness scores for each function evaluation in the optimisation process.

Fig.11a shows a confusion plot generated from the pre-finetuning test on the CIFAR-10 dataset, with the raw confusion matrix shown below.

933	0	119	24	4	819	6	20	12	2
18	15	2	4	0	1	0	0	965	947
0	1	0	0	0	0	0	0	0	4
1	963	30	94	22	122	990	4	8	17
0	0	0	0	0	0	0	0	0	0
12	21	834	15	14	6	1	20	9	12
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
32	0	5	846	7	39	1	19	6	16
4	0	10	17	953	13	2	937	0	2

Fig.11b shows the confusion plot generated after fine-tuning, with the accompanying raw confusion matrix shown below.

965	1	7	6	3	2	3	4	16	4
1	979	0	1	0	0	0	0	5	22
13	0	934	18	8	9	8	4	5	1
2	0	14	878	9	41	8	8	2	1
1	1	15	16	966	12	3	12	0	0
0	0	13	64	8	930	1	9	0	1
1	0	10	7	2	3	975	1	0	0
0	0	3	5	3	2	2	961	0	0
16	3	3	4	1	1	0	0	968	5
1	16	1	1	0	0	0	1	4	966

It can be seen that the fine-tuning process is effective in reversing the overfitting on the training data and allows the optimal CNN to generalise to greatly improved performance on the test data. This is owing to the robustness of the weights learned through our combined optimisation and training process.

The CIFAR-100 dataset consists of the same number of images but split over 100 classes, each of which belongs to one of twenty ‘superclasses’. Each class has 500 training images and 100 testing images, resulting in the same training/testing split as that of CIFAR-10 (50,000 vs 10,000 respectively). As with CIFAR-10, training images are further split into 45,000 training and 5,000 validation, and the validation images are used to generate the model fitness after each optimisation function evaluation.

Fig. 12 shows some analysis of the lookup table following the final access during the optimisation/ training process. Each line represents a different block in the architecture, with the number of layers in the block displayed along the x axis. The y axis shows the lowest error rate achieved by an architecture with that configuration of block in its architecture (although the other blocks could be in any configuration). From

the CIFAR-10 experiment, the embodiment tends to prefer more depth in the initial blocks, whilst the feature maps are larger and the receptive field is smaller, and more shallow blocks later in the network, especially when it comes to the fully connected layers. This becomes drastically more pronounced in the CIFAR-100 experiment when the number of output classes is increased by an order of magnitude and the same pattern can still be seen.

Figure 13 shows a method 1300 of generating and using a CNN ensemble according to an embodiment. An ensemble is a plurality of neural networks (eg CNN) which may be used to enhance classification accuracy compared with a single CNN. For example, an image may be classified by each NN in the ensemble and the resulting classifications arbitrated in some manner in order to arrive at a more accurate result. This may be done by majority or plurality voting, for example if 5 NN classify an image as containing a dog, 2 NN classify the image as containing a cat, and 1 NN classify the image as containing a horse, the ensemble is judged to have classified the image as containing a dog.

At step 1305 of the method 1300, the ensemble is generated from the local best candidate CNN already determined in the previously described methods, including the global best or optimal candidate CNN. As described previously, for example with respect to Figure 6, each particle converges on a local best of architectural parameters. Because these parameters from all of the local best CNN candidates have already been calculated and stored when running the CNN generation method, for example as described with respect to Figures 2 – 12, no additional computational resources or processing is required to generate the local best CNN candidates. Some of the local best particles will have converged to the same position. In this example one representative particle or candidate CNN is used from each distinct position which improves speed, however in other examples all local best including those with the same particle position may be used to form the ensemble.

At steps 1310 and 1315, each local best CNN candidate is further trained or fine-tuned. The CNN retains the weights determined from the earlier CNN generation method as its initial weights and is then fine-tuned by further trained on the training and validation data combined for a limited number of epochs. This process does not require significant further processing. At step 1320, an image for classification is received.

This step may be performed independently of the ensemble generation steps 1305 – 1315, for example at a later time or following receipt of the ensemble from a remote or online source. At step 1325 the image is processed through each CNN of the ensemble to determine its respective classification of the image.

At step 1330, a classification of the image from the ensemble is determined. This may be done using plural or majority voting for example, so that the classification determined by the majority of CNN is allocated as the classification of the ensemble. Various other methods of arbitrating between different classifications from the individual CNN may alternatively be employed.

Experimental data can be seen in Figure 18 and the table below. Each local best particle position has a corresponding accuracy and error rate. The particle positions correspond to local best CNN candidates with the shown number of layers (the architectural parameter in this example) in each block of the predetermined architecture. It can be seen that the Ensemble improves on the accuracy of individual candidates as well as their error rates, including the global best. Figure 18 shows each distinct local best position as a projection on a 2D space using t-Distributed Stochastic Neighbour Embedding (t-SNE).

Particle Position	Particle Tally	Accuracy (%)	Error Rate (%)
[5, 3, 3, 0, 0] (Global Best)	38	95.34	4.66
[5, 2, 2, 0, 0]	4	95.16	4.84
[4, 2, 3, 0, 0]	1	95.13	4.87
[5, 3, 2, 0, 0]	5	95.29	4.71
[5, 2, 3, 0, 0]	2	95.28	4.72
Ensemble	N/A	95.61	4.39

Figure 17 shows a method 1700 of generating and using a CNN ensemble according to another embodiment. At step 1705 of the method 1700, each block B_i is considered individually, and the fitness score values associated with the candidate CNN for each block B_i are retrieved from the weight sharing lookup table. For each block B_i the two (or more) best fitness values seen are used to select two block configurations (b_1, b_2). In this way a set of tuples in B can be built up eg $B = \{(1,2),(1,4),(6,3),(2,7),(0,1)\}$.

At step 1710, new CNN candidates are generated using combinations of blocks from the identified two best blocks for each block position in the architecture. The candidates can be generated by taking the cartesian product of all tuples:

$$A = B_1 \times \dots \times B_n = \{(b_1, \dots, b_n) | x_i \in X_i \forall i \in \{1, \dots, n\}\}$$

All possible, or a sub-set of, combinations of the best two blocks are used to generate CNN for the new ensemble. For example, for a predetermined architecture having 5 blocks, and taking the two best candidate blocks, there are 2^5 or 32 combinations of blocks that can be used as the generated new CNN. An individual model may be represented by $A_i = [a_1, a_2, a_3, a_4, a_5]$, e.g. the first model nominated from B above will be $A_1 = [1, 1, 6, 2, 0]$. Because all of the weights and other parameters for the blocks have already been calculated and stored when running the CNN generation method, for example as described with respect to Figures 2 – 12, significant additional computational resources or processing is not required. In some examples, candidate CNN may be optimised using these methods for different architectural parameters. The two best blocks for each parameter may then be used with the cartesian product process above to generate more CNN candidates.

At steps 1715 and 1720, each generated new CNN candidates are further trained or fine-tuned. The CNN retains the weights determined from the earlier CNN generation method as its initial weights and is then fine-tuned by further training on the training and validation data combined for a limited number of epochs. This process does not require significant further processing.

At step 1725, an image for classification is received. This step may be performed independently of the ensemble generation steps 1705 – 1720, for example at a later time or following receipt of the ensemble from a remote or online source. At step 1730 the image is processed through each CNN of the ensemble to determine its respective classification of the image.

At step 1735, a classification of the image from the ensemble is determined. This may be done using plural or majority voting for example, so that the classification determined by the majority of CNN is allocated as the classification of the ensemble. Various other methods of arbitrating between different classifications from the individual CNN may alternatively be employed.

Experimental data can be seen in Figure 19 and the table below. Each local best particle position has a corresponding accuracy and error rate. The particle positions correspond to new CNN candidates with the shown number of layers (the architectural parameter in this example) in each block of the predetermined architecture. It can be seen that the Ensemble improves on the accuracy of individual candidates as well as their error rates, including the global best. Figure 19 shows the positions of the candidate models derived from the look-up table method - each position is shown as a projection on a 2D space using t-Distributed Stochastic Neighbour Embedding (t-SNE).

(Integer) Position	Accuracy (%)	Error Rate (%)
[5, 3, 3, 0, 0] (Global Best)	95.37	4.63
[5, 3, 3, 0, 1]	95.36	4.64
[5, 3, 3, 1, 0]	95.42	4.58
[5, 3, 3, 1, 1]	95.32	4.68
[5, 3, 2, 0, 0]	95.25	4.75
[5, 3, 2, 0, 1]	95.30	4.70
[5, 3, 2, 1, 0]	95.27	4.73
[5, 3, 2, 1, 1]	95.25	4.75
[5, 2, 3, 0, 0]	95.27	4.73
[5, 2, 3, 0, 1]	95.22	4.78
[5, 2, 3, 1, 0]	95.19	4.81
[5, 2, 3, 1, 1]	95.23	4.77
[5, 2, 2, 0, 0]	95.20	4.80
[5, 2, 2, 0, 1]	95.14	4.86
[5, 2, 2, 1, 0]	95.12	4.88
[5, 2, 2, 1, 1]	95.17	4.83
[4, 3, 3, 0, 0]	95.17	4.83
[4, 3, 3, 0, 1]	95.05	4.95
[4, 3, 3, 1, 0]	95.21	4.79
[4, 3, 3, 1, 1]	95.21	4.79
[4, 3, 2, 0, 0]	95.14	4.86
[4, 3, 2, 0, 1]	95.10	4.90

[4, 3, 2, 1, 0]	95.09	4.91
[4, 3, 2, 1, 1]	95.09	4.91
[4, 2, 3, 0, 0]	95.25	4.75
[4, 2, 3, 0, 1]	95.28	4.72
[4, 2, 3, 1, 0]	95.11	4.89
[4, 2, 3, 1, 1]	95.16	4.84
[4, 2, 2, 0, 0]	95.10	4.90
[4, 2, 2, 0, 1]	94.99	5.01
[4, 2, 2, 1, 0]	95.02	4.98
[4, 2, 2, 1, 1]	95.04	4.96
Ensemble	95.73	4.27

A practical application of the above described methods is described with reference to Figures 14 – 17 which illustrate a media post-production application. Whilst sound may be recorded with video in the production of movies, television shows, advertisements, video games and other video segments or tracks, the sounds often need to be enhanced, augmented or changed in order to produce a video track acceptable to a viewer or audience. It is difficult to obtain the required sound fidelity, accuracy or impression using recorded sound alone, and therefore post-production may add to or replace some recorded sound with separately recorded sound for example generated and pre-recorded by Foley actors. This may include the addition of background sound corresponding to different scenes such as a pub or a desolate forest. This may also include the addition of sounds based on events or action within a video sequence, such as footsteps, breaking glass or a door closing. This is typically a very manual, time consuming and costly process. In an embodiment, this process may be partially automated with the use of neural networks (or ensembles of neural networks) generated using the above described (or different methods).

A media post-production system according to an embodiment is illustrated in Figure 16. The system 1600 includes a processor 1605, a server 1610 and a user interface 1615 and one or both of a background sound algorithm 1655 and a neural network generation algorithm 1675. The NN generation algorithm 1675 may be implemented using any of the previously described methods. The background sound algorithm 1655 is described in more detail below with reference to Figure 15. The

processor may be any suitable computing resource and the server 1610 any suitable memory resource including local and/or remote storage.

The NN generation algorithm 1675 may be implemented locally by the system 1600, for example using training images 1670 in order to generate one or more CNN 1635. These may be used to generate CNN that identify many different objects such as different actors, animals or scenes such as a pub, a street, a quiet meadow, a windy forest and so on. Alternatively, the CNN may be downloaded or otherwise retrieved from an external source or supplier.

A plurality of different CNN 1635 generated and trained for different classifications may be used to recognise different scenes and/or objects. Different objects may be recognised and localised as illustrated by dividing an image 1620 into multiple regional propositions or parts 1625 in order to recover sub-images 1630 which can then each be fed into an array of CNN 1635 and classified – graph detail 1640 indicates the confidence levels for different classifications for one of the sub-images 1630. In an example application, the image 1620 may correspond to one frame in a sequence of frames comprising a video segment or track. The regional proposition technique may be used together with the CNN to track an object such as a dog moving across the frames. The sub-image classified as containing the dog in each frame will change as the dog moves within the video. This tracking can be used to adjust sound (eg barking or footfalls) associated with the dog – for example the amplitude of the sound in left and right audio channels may be automatically adjusted as the dog moves across the frames.

In another example, CNN may be used to recognise a scene such as a crowded pub or bar in the frames 1620 making up a video track 1645. The track 1645 may include a recorded sound file 1650 temporally aligned with the frame sequence. The background sound algorithm 1655 may use CNN 1635 to automatically identify a scene associated with the frames and add a background sound file 1665 to those frames. If the scene changes, this will be recognised by the plurality of CNN – for example a pub scene trained CNN no longer indicates a pub scene and instead a desolate forest trained CNN indicates a desolate forest. A different sound file 1665 may then be associated with the new frames 1620.

Figure 14 illustrates a screen capture 1400 from the user interface 1615. The screen 1400 includes various functional elements 1405 such as file selection, frame sequence, current frame display, detail portion of current frame, and various controls. Many other or alternative functional elements are also possible. A pop-up window 1410 includes options for operating the background sound algorithm 1655. This may include timecodes within the sequence of frames, for example precise durations from the beginning of the sequence and which may correspond to a sub-sequence of frames for analysis. The frame rate may be selected, as well as the processing required, for example determination of the scene or atmosphere and determination of actions or events such as footsteps.

Figure 15 illustrates a method 1500 of adding background sound to a sequence of frames, for example as used by the background sound algorithm 1655. At step 1505, a sequence of frames is received from a video track, for example using the pop-up window from Figure 14. The frames will be associated with respective timecodes and may correspond to images from a movie segment. At steps 1510 and 1515, the images or frames are classified by a plurality of CNN, for example as being from a bar or forest scene.

At step 1520, the first and last frames having the same classification are identified, for example a sequence of frames identified as being from a bar scene. At step 1525 a sound file or clip (eg a .wav file) corresponding to a bar scene is retrieved, for example locally or downloaded from a remote library of background sound files. At step 1530, the sound file is associated or added into the track at the timecode corresponding to the first frame having the classification and for a duration or until the timecode of the last frame having the same classification. At step 1535, it is determined whether there are frames in the sequence which are classified differently and if so, a new sequence having the same classification is determined and a corresponding sound file associated with these in the enhanced video track 1660. At step 1540, the enhanced track may be stored for later use or further processing.

Models or NN generated by the embodiments may also be used in commercially available tools used for various media production, security or medical purposes. For example, an NN optimised/trained to recognise a person may be used with a tool used to determine footstep events or the gait of that person over a number of frames.

Identification of footstep events may be used to automatically add sound to a video track. The gait information may be used to add fabric swishing sounds for example.

The gait information may be compared with predetermined gait information corresponding to a particular person in order to recognise that particular person from their gait. This gait matching may be used for security applications such as unlocking a door. This matching may also be used to determine differences between a particular person's predetermined gait and their current gait, which may be used for medical analysis, for example to determine whether the person has had a stroke. This may be a useful system for deployment in a retirement village for example.

An example of a commercially available systems that can be used to determine gait, identify footsteps or other events include Pro Tools Media Composer from Avid at www.avid.com.

The above embodiments are to be understood as illustrative examples of the invention. Further embodiments of the invention are envisaged. It is to be understood that any feature described in relation to any one embodiment may be used alone, or in combination with other features described, and may also be used in combination with one or more features of any other of the embodiments, or any combination of any other of the embodiments. Furthermore, equivalents and modifications not described above may also be employed without departing from the scope of the invention, which is defined in the accompanying claims.

CLAIMS

1. A computerised image classification method, comprising:
generating successive candidate neural networks (NN) using an optimisation algorithm, each NN having a number of connected blocks of layers, the layers having a plurality of neurons with connections having associated weights;
wherein each block comprises fixed and variable architectural parameters, the or each variable architectural parameter being determined by an optimisation algorithm;
training each candidate NN using training data in order to update the weights of the candidate NN, and determining a fitness function score of the trained candidate NN using validation data;
wherein if there is a block having the same architectural parameters from a previously trained candidate NN, inheriting the weights associated with layers of said block prior to training and fitness score determination;
selecting one of the candidate NN based on the respective fitness function scores and classifying an image using the selected NN.
2. The method of claim 1, further comprising associating a sound file with the image dependent on the classification.
3. The method of claim 2, further comprising determining a sequence of images having the same classification, the images having respective timecodes in a video file; wherein associating the sound file comprises associating a portion of the sound file having a duration corresponding to the duration between the timecodes of the first and last image in the sequence.
4. The method of any one preceding claim, wherein the blocks of layers are arranged into a predetermined architecture with each block having a respective location within the predetermined architecture, and wherein weights are inherited only from blocks having the corresponding location within the predetermined architecture of a previously trained NN.

5. The method of claim 4, wherein each block is of a predetermined block type having a number of predetermined layers and a number of variable layers dependent on the architectural parameter of the block.
6. the method of claim 5, wherein the NN is a convolutional neural network and the variable layers comprise groups of the following layers: convolution; batchnorm; ReLU.
7. The method of any one preceding claim, wherein the architectural parameter comprises one or more of the following: number of layers; filter sizes; filter depths; number of filters; filter strides; filter paddings; filter biases; filter dilations; connections between layers; type of convolution.
8. The method of claim 7, wherein the weights to be inherited are determined according to a non-linear function from a corresponding block of the last or best previous candidate NN having said corresponding block, the best previous NN determined by its fitness function score.
9. The method of any one preceding claim, wherein the optimisation algorithm is a particle swarm optimisation (PSO) algorithm, each particle corresponding to the architectural parameter of each block in the candidate NN.
10. The method of claim 9, wherein the PSO comprises acceleration coefficients which are adapted over the duration of a search according to a non-linear function.
11. The method of any one preceding claim, wherein an optimal NN is determined dependent on the fitness function scores of the candidate NN, retains the weights from the training step and is then further trained using the training data and/or validation data.
12. The method of any one preceding claim, wherein the NN is optimised and trained to recognise one or more of the following: an object in an image; the scene of a

particular image; an anomaly in a medical image; an event in a visual sequence of images; a person's face or gait.

13. The method of claim 12, wherein the NN is optimised and trained to determine a medical diagnosis in response to recognising the anomaly in the medical image.
14. The method of claim 12, further comprising highlighting the anomaly in the medical image by cropping the image around the anomaly or colouring the anomaly.
15. The method of claim 12, wherein the NN is optimised and trained to recognise an event in a visual sequence and in response to automatically initiate an action.
16. The method of claim 13, wherein the action is one or more of the following: change the mode of a security system; send a medical alert; generate an onscreen menu of options for changing audio data associated with the recognised event; changing audio data associated with the recognised event.
17. The method of any one preceding claim, comprising using additional NN determined from the generated candidate NN to generate an ensemble of NN to classify the image.
18. The method of claim 17, wherein the additional NN are determined using the best local fitness score for each candidate.
19. The method of claim 17 comprising:
 - determining the best two blocks for each block position in the architecture from the candidate NN;
 - generating an ensemble of additional NN by combining different combinations of the determined blocks;
 - using the ensemble to classify the image.

20. An image classification apparatus, comprising:
a memory and a processor which when executing instructions stored on the memory is arranged to perform the method of any one preceding claim.
21. The apparatus of claim 20, wherein the NN is a convolutional neural network.
22. A media post-production apparatus for processing a sequence of images, the apparatus comprising:
a processor and memory to implement a plurality of NN each capable of a different classification of an image;
a classification engine to use the NN to classify one or more of the images;
a sound engine to associate a sound with the images depending on their classification;
the classification engine comprising the image classification apparatus of claim 20 or 21.
23. The apparatus of claim 22, further comprising a training engine to generate and train an additional NN.
24. The apparatus of claim 22 or 23 wherein the sound is a background sound file and:
the classification engine to determine a sequence of images having the same classification, the images having respective timecodes in a video file;
the sound engine to associate a portion of the sound file having a duration corresponding to the duration between the timecodes of the first and last image in the sequence.

1/18

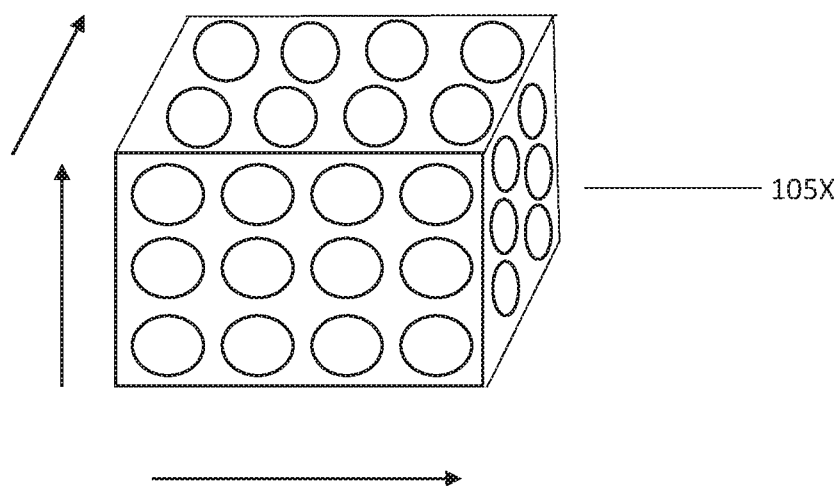
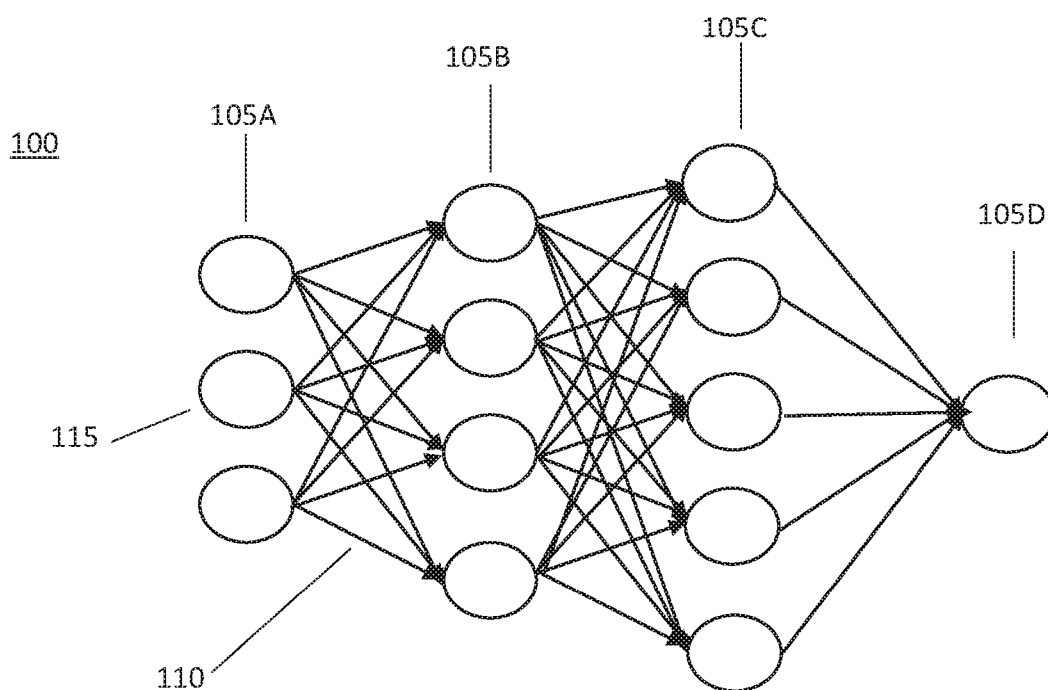


FIG. 1

2/18

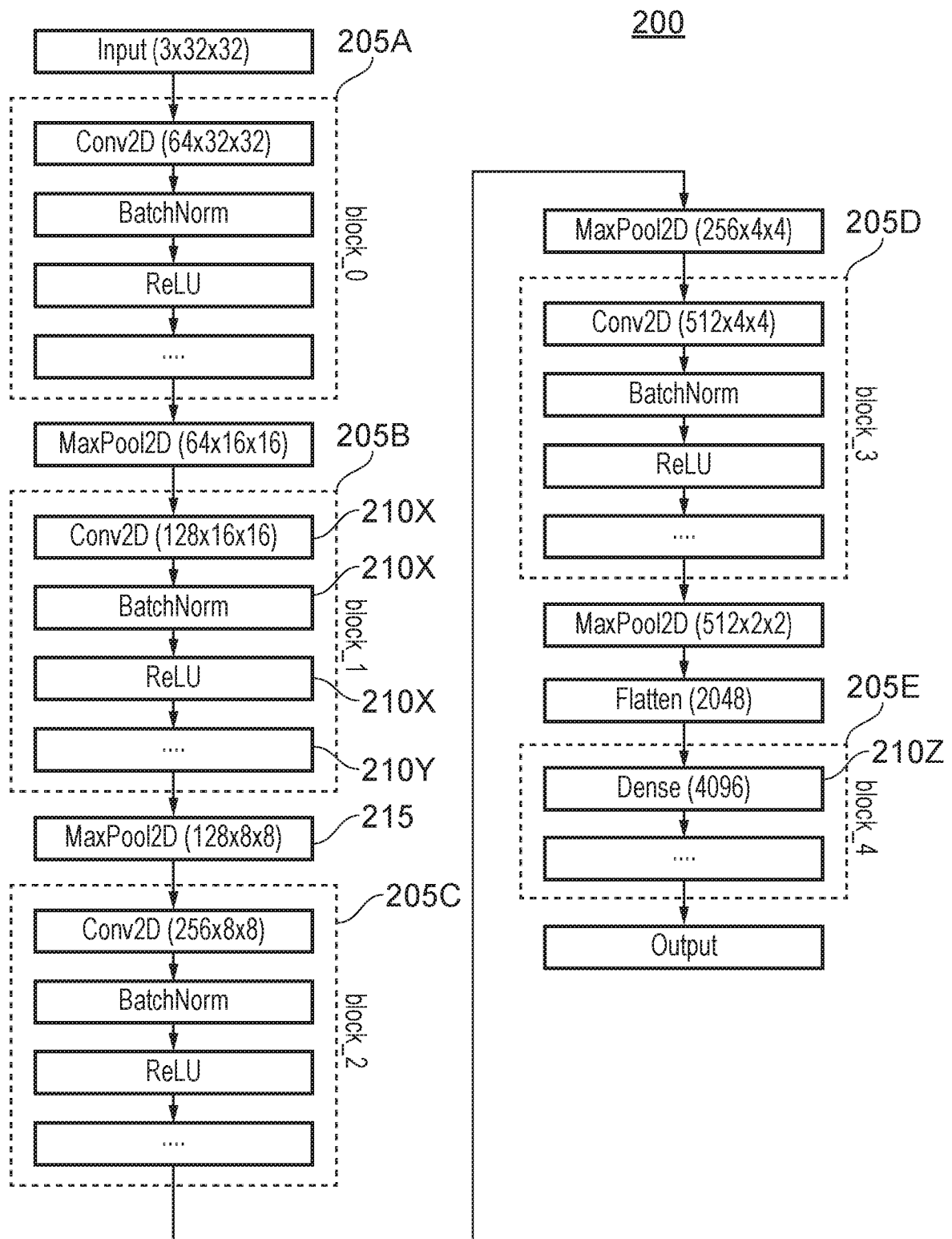


FIG. 2

3/18

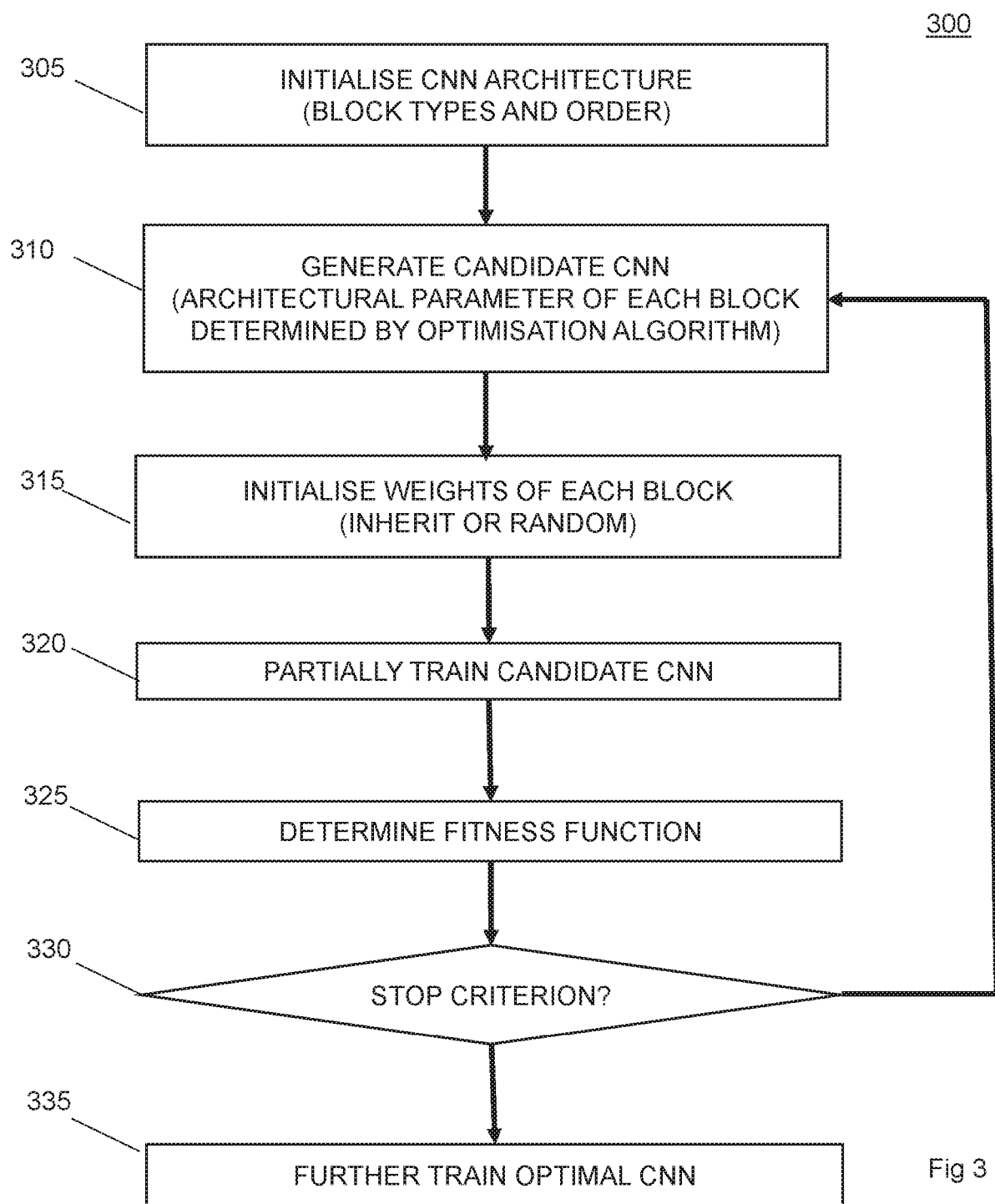


Fig 3

FIG. 3

4/18

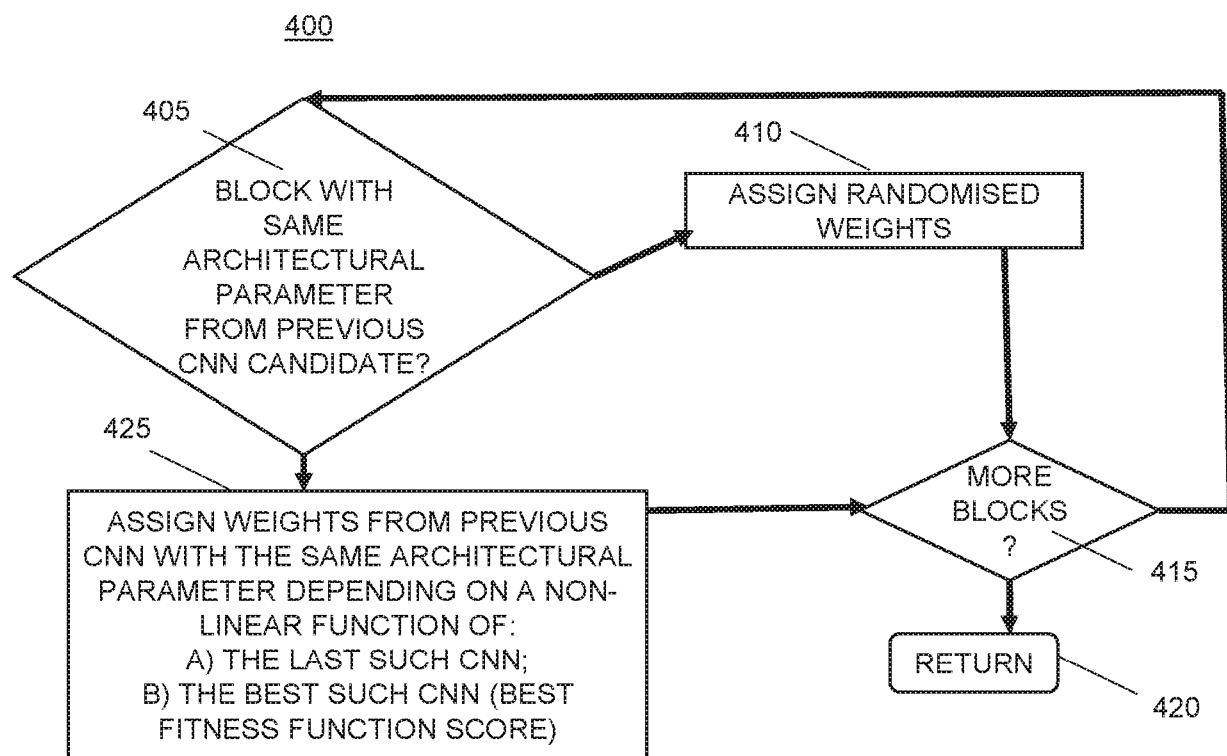


FIG. 4

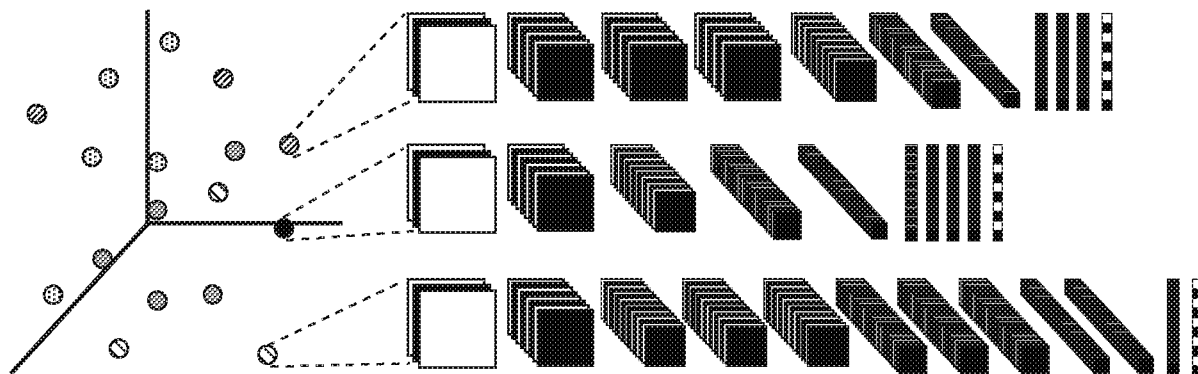


FIG. 6

5/18

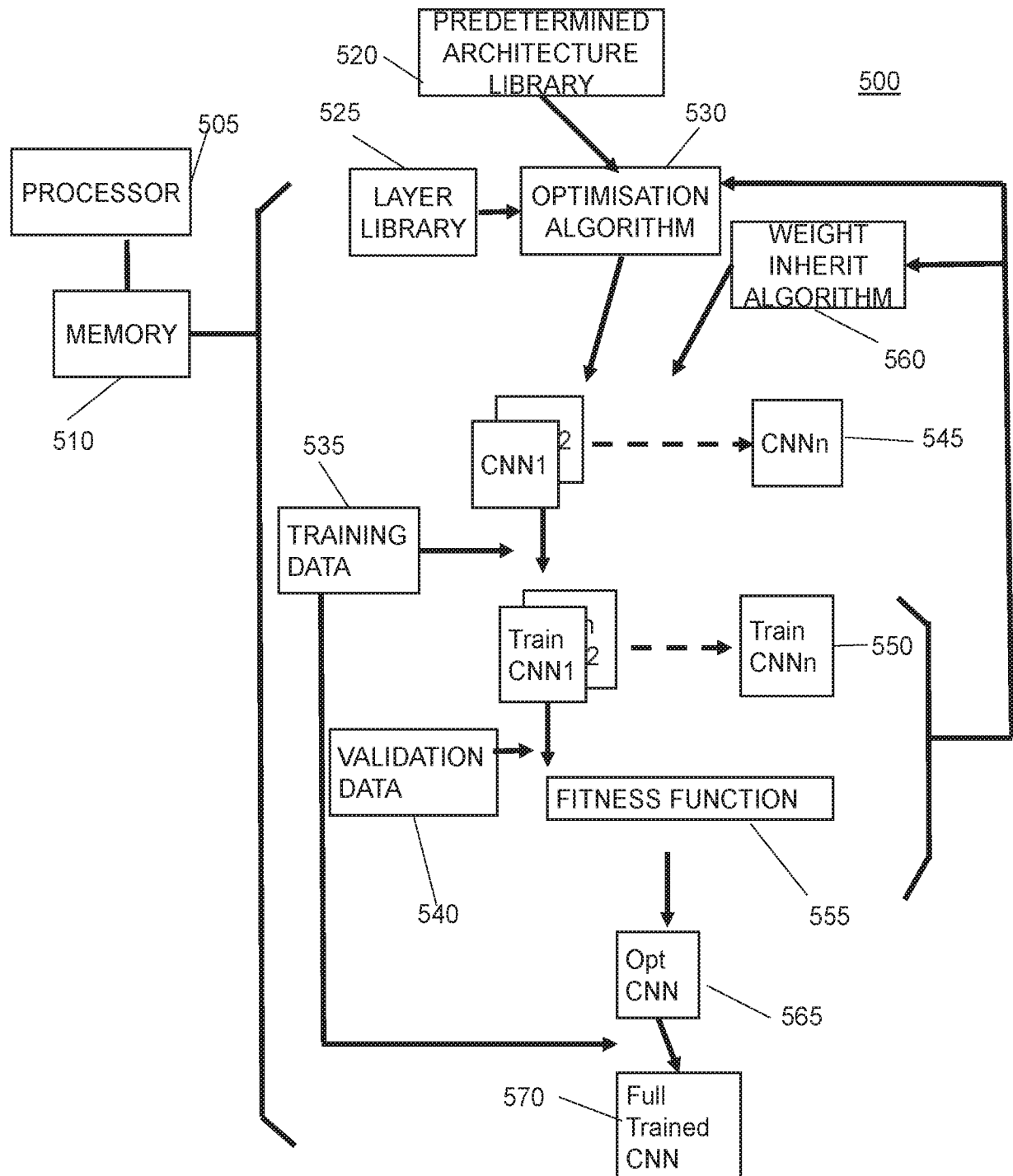


FIG. 5

6/18

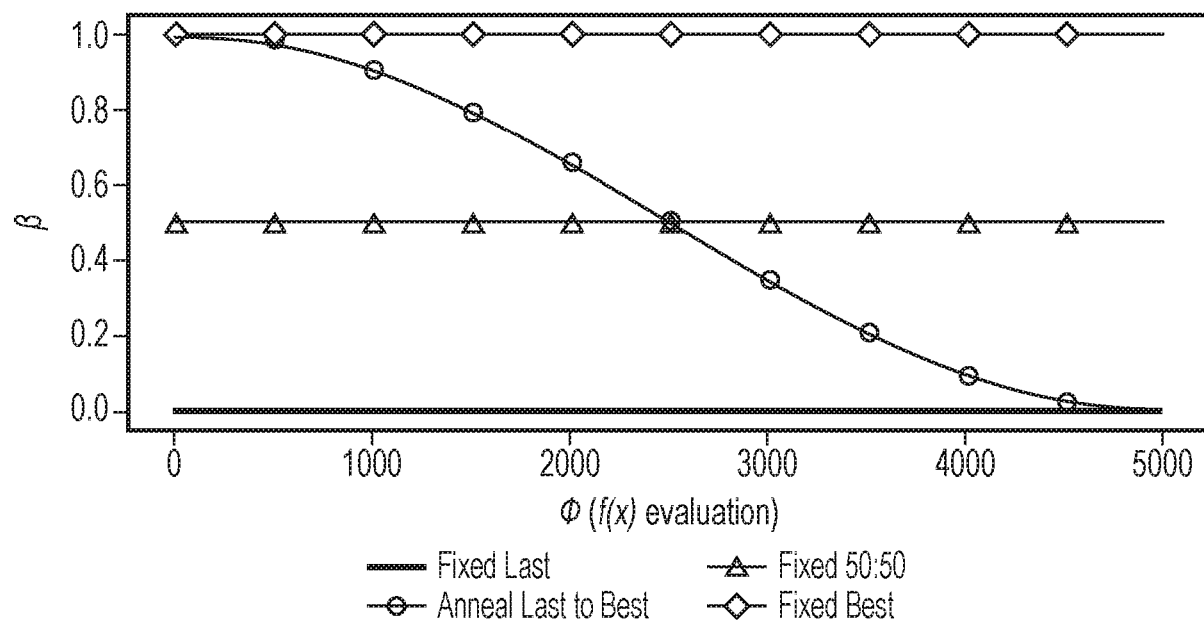


FIG. 7

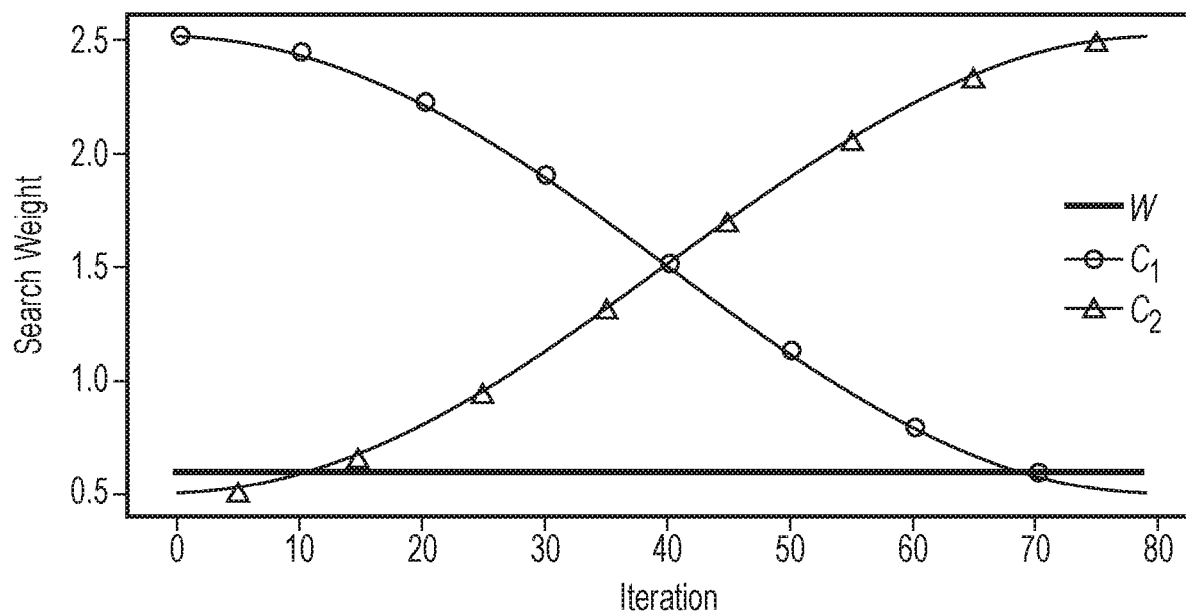


FIG. 8

7/18

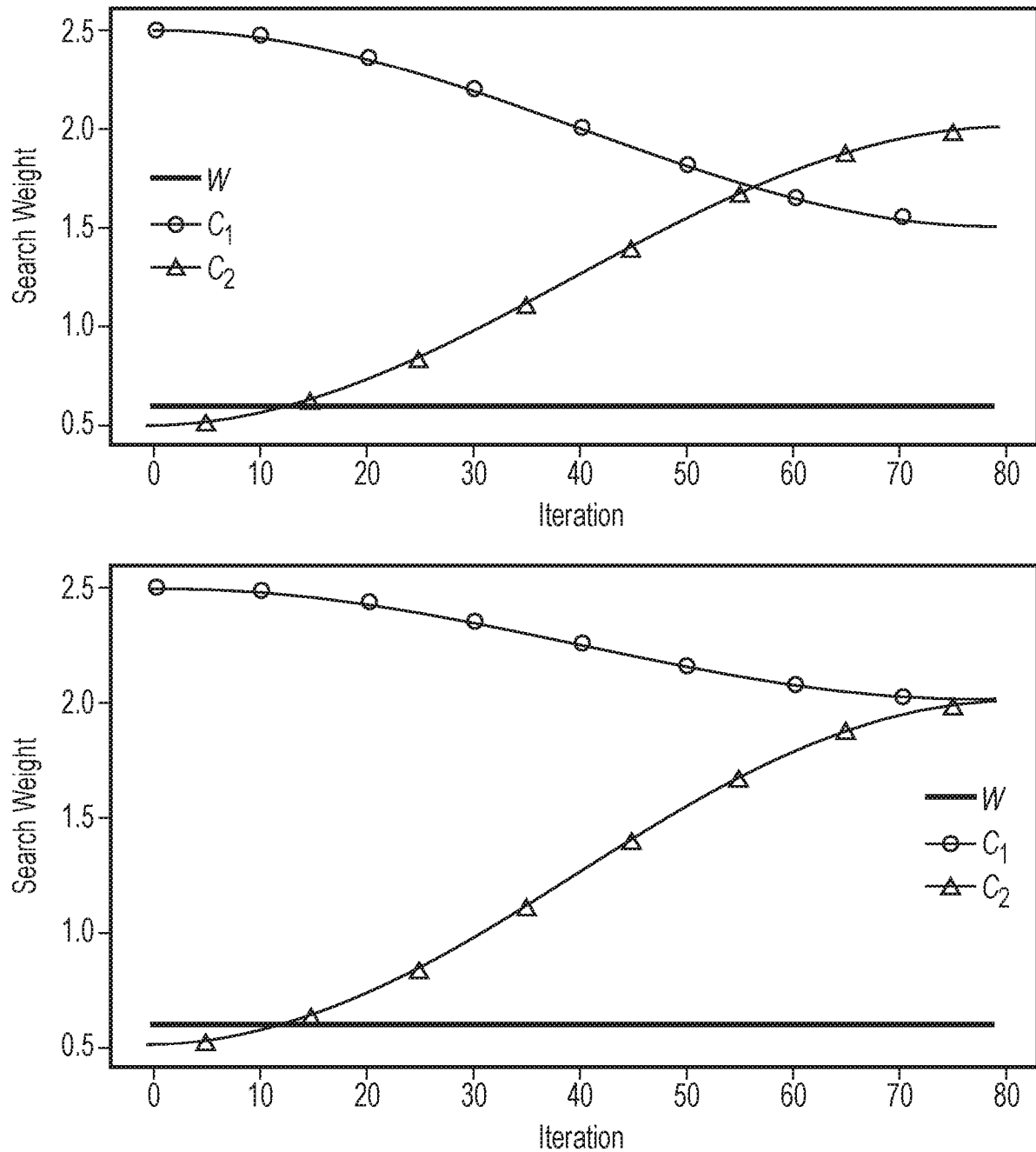


FIG. 8 (Continued)

8/18

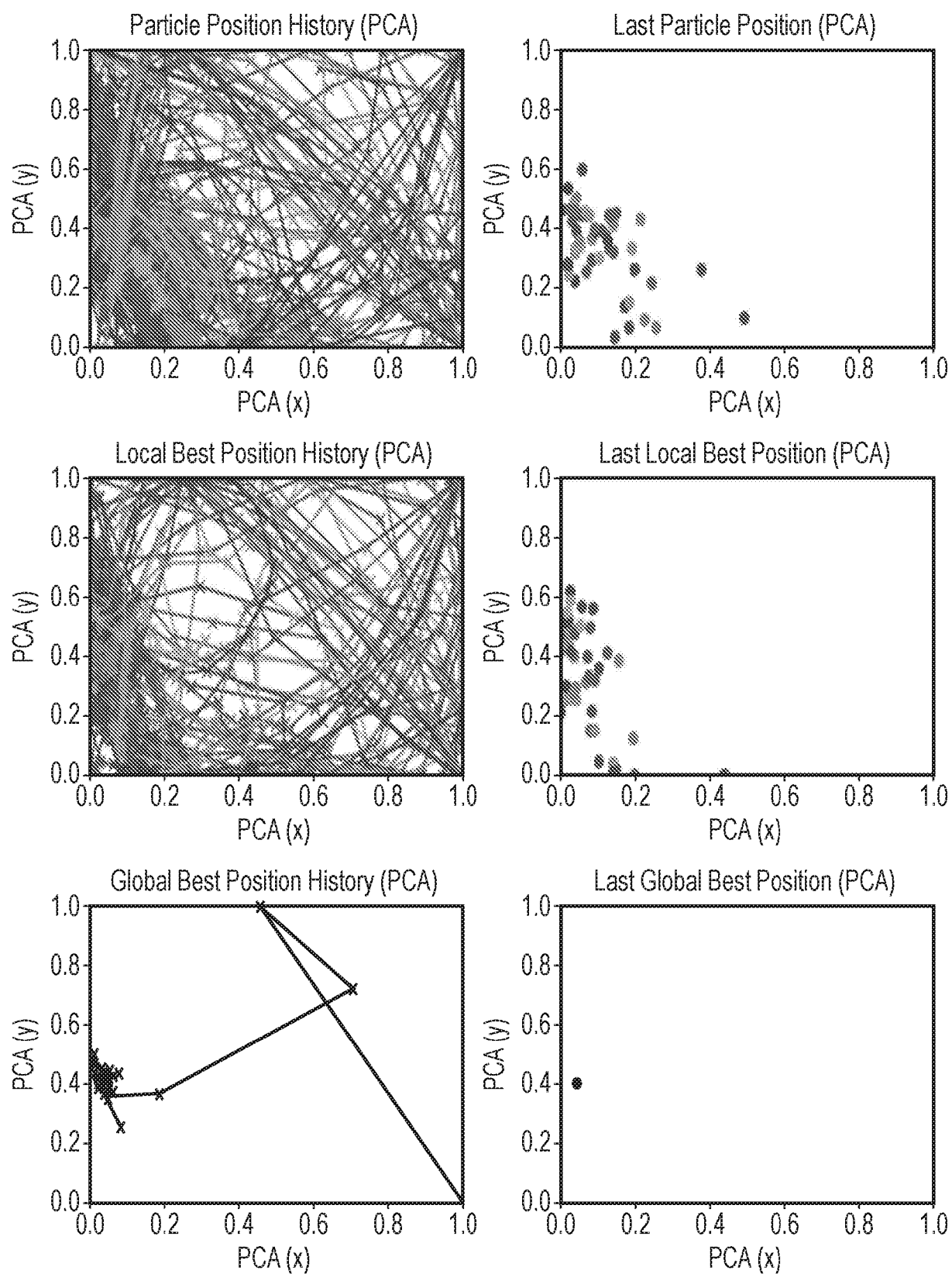
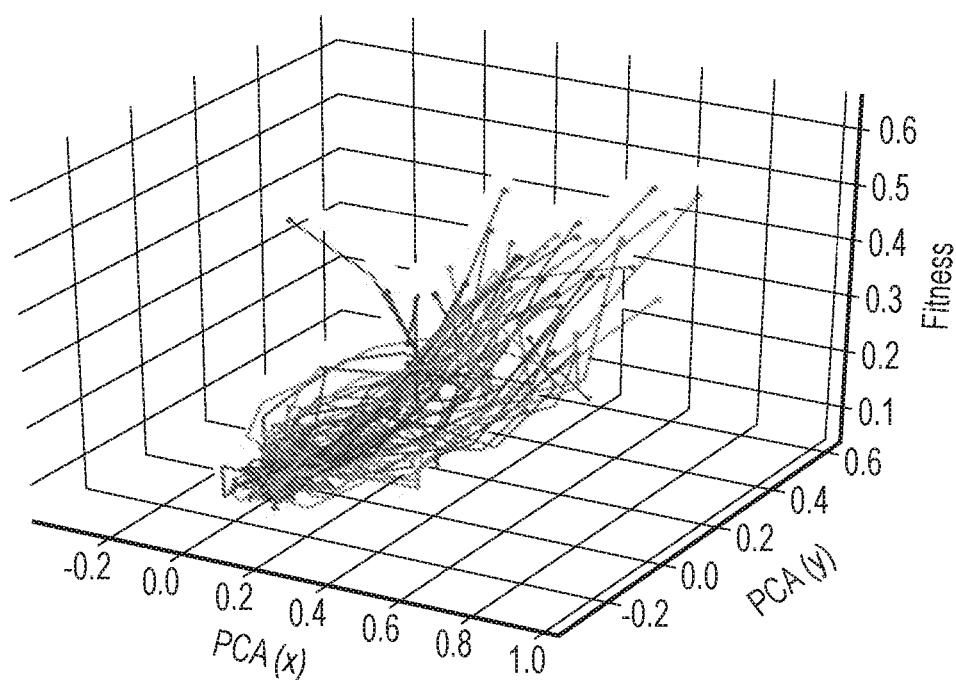
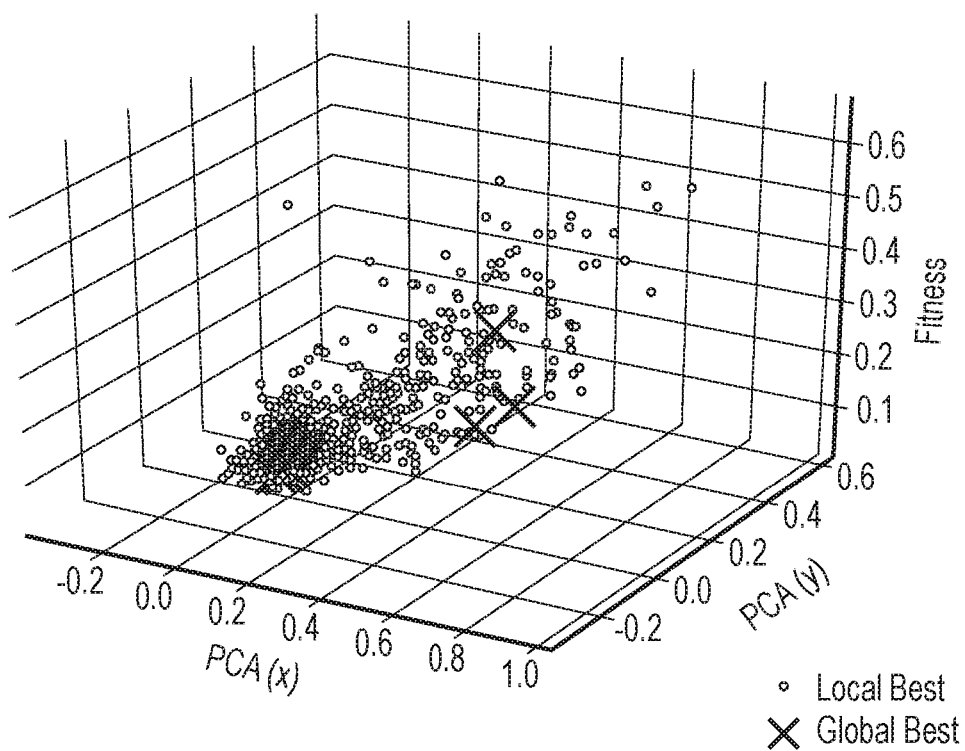


FIG. 9

9/18



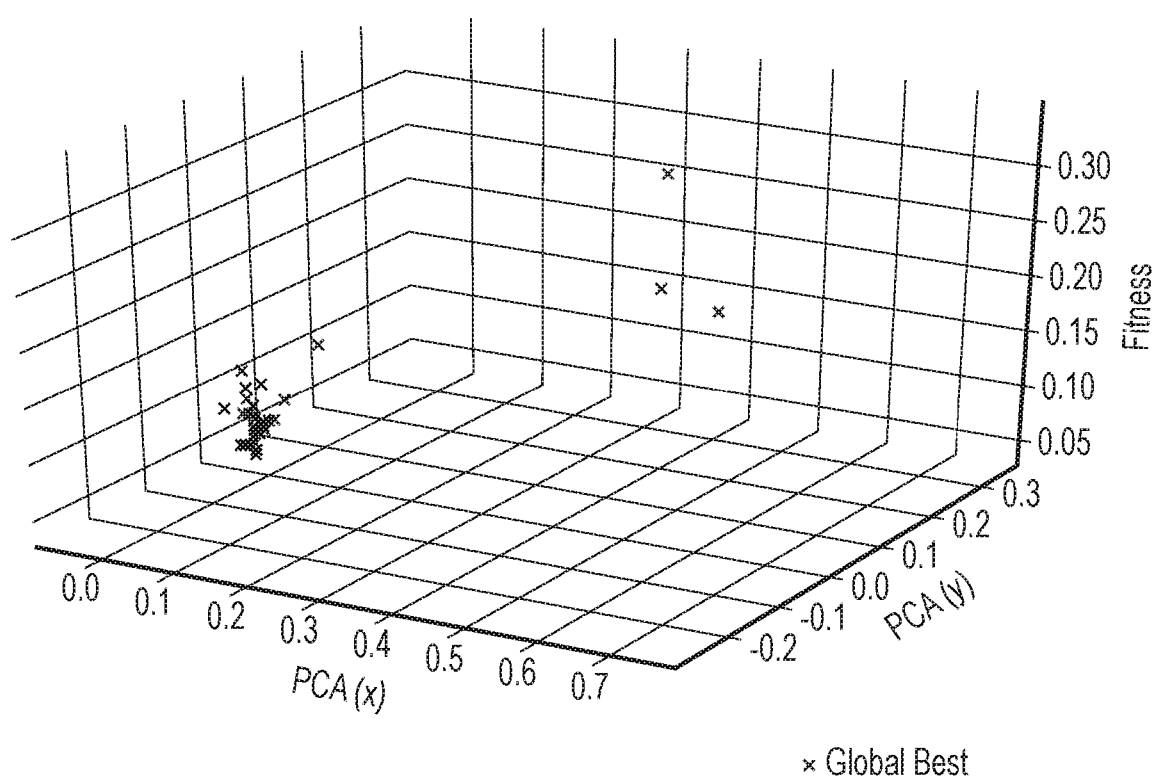
(a) Local best positions & their fitness scores by iteration for all 50 particles (point indicates start position)



(b) All local & global best positions & their fitness scores

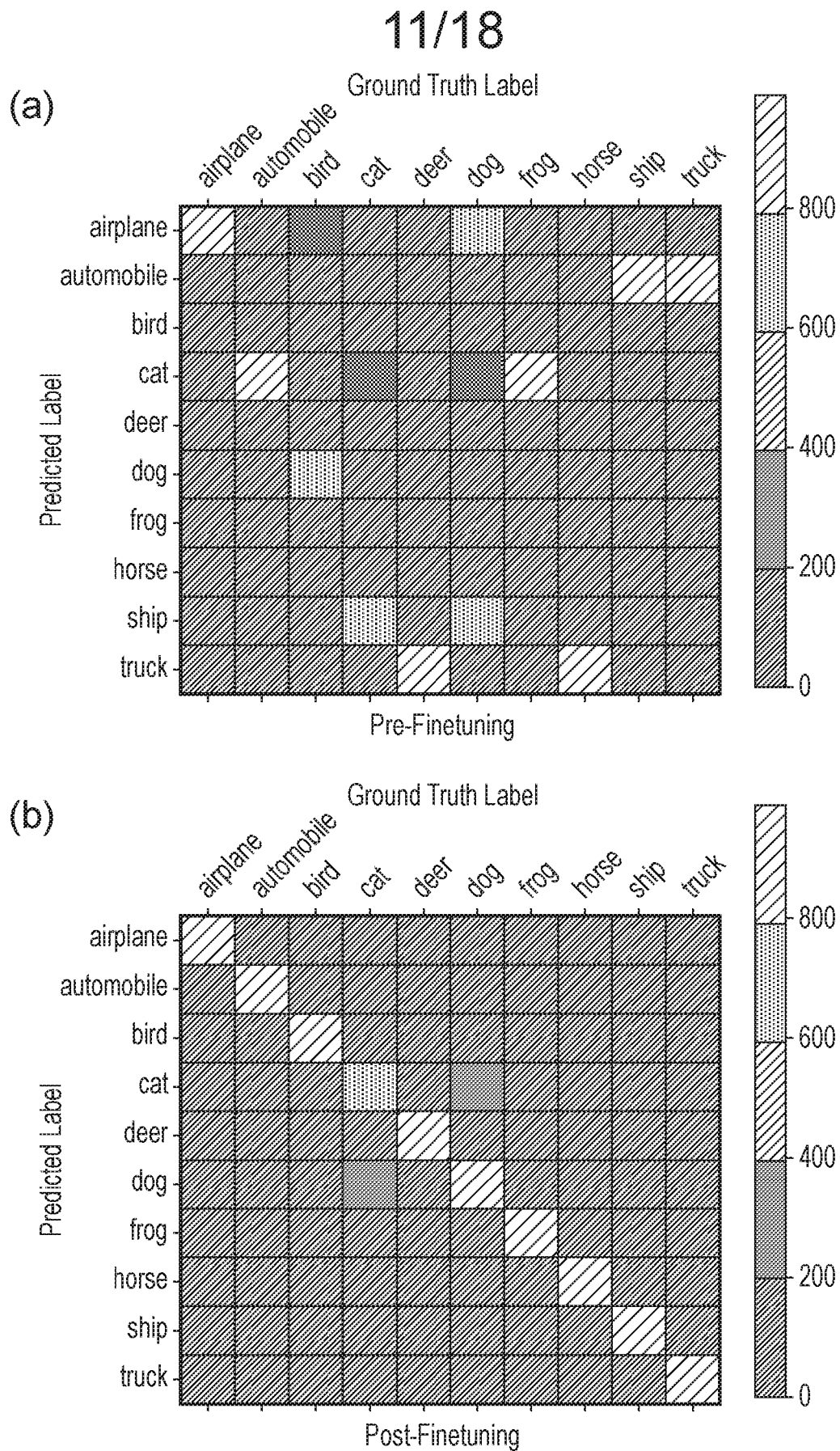
FIG. 10

10/18



(c) Global best positions & their fitness scores

FIG. 10 (Continued)



12/18

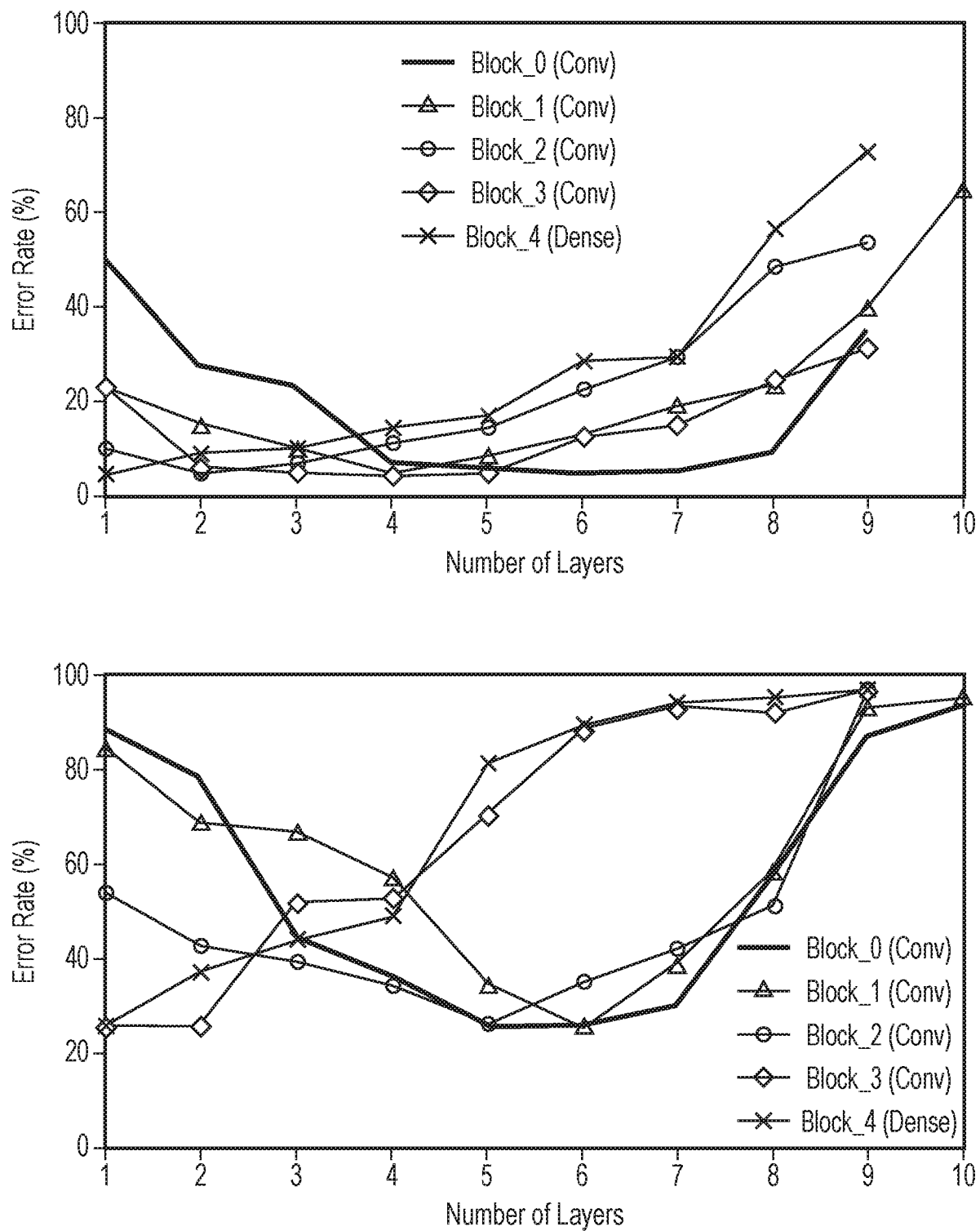


FIG. 12

13/18

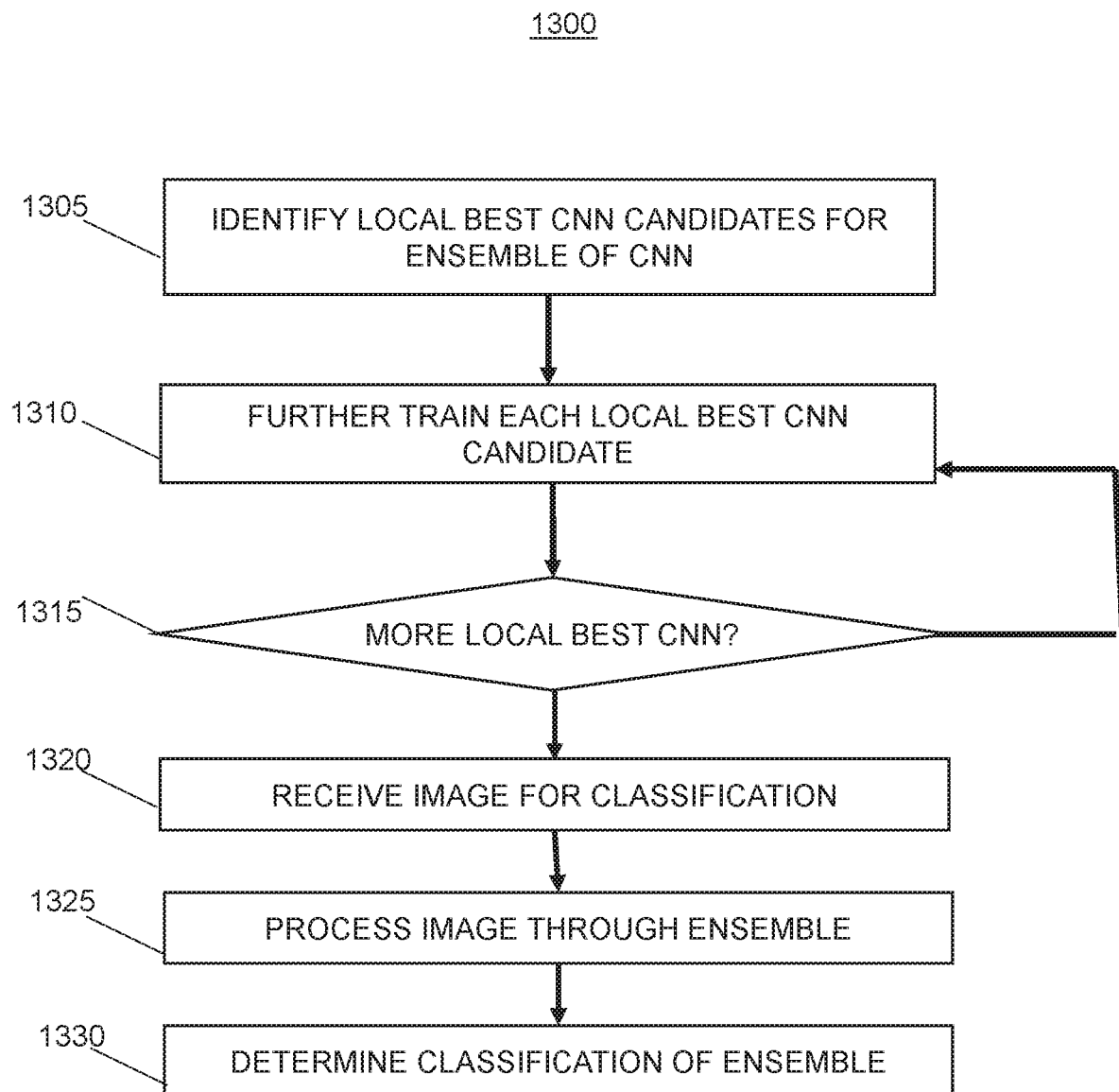


FIG. 13

14/18

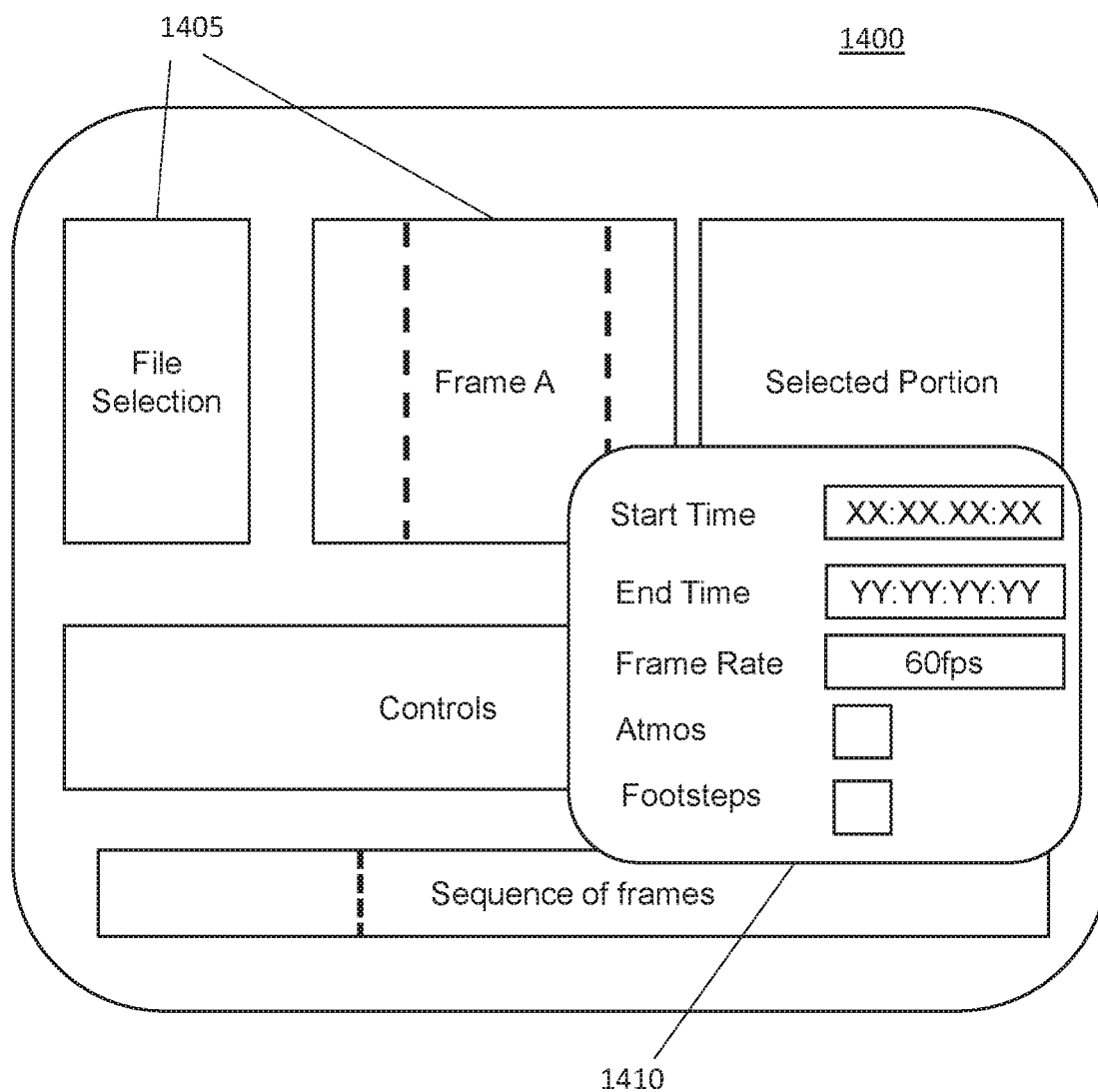


FIG. 14

15/18

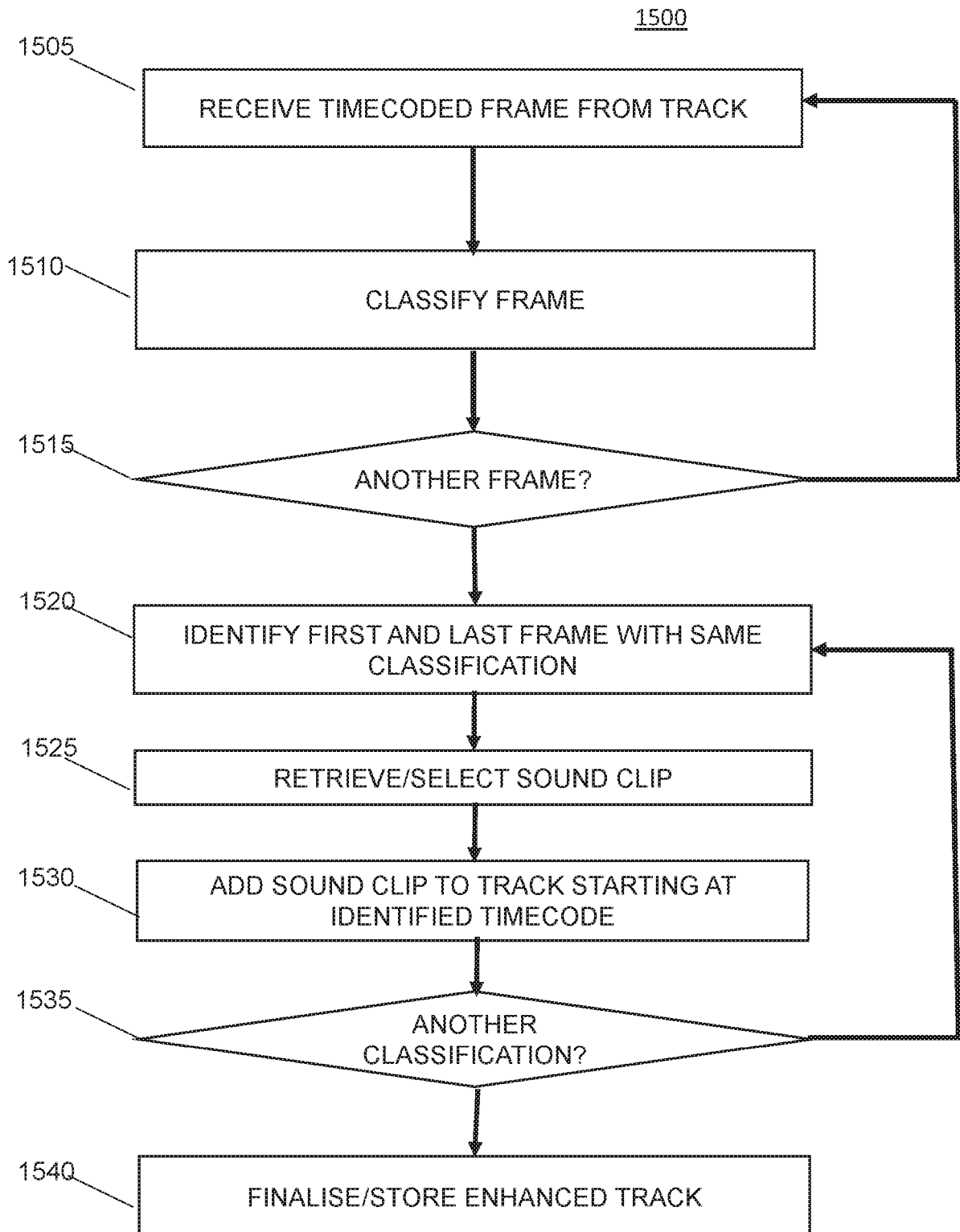


FIG. 15

16/18

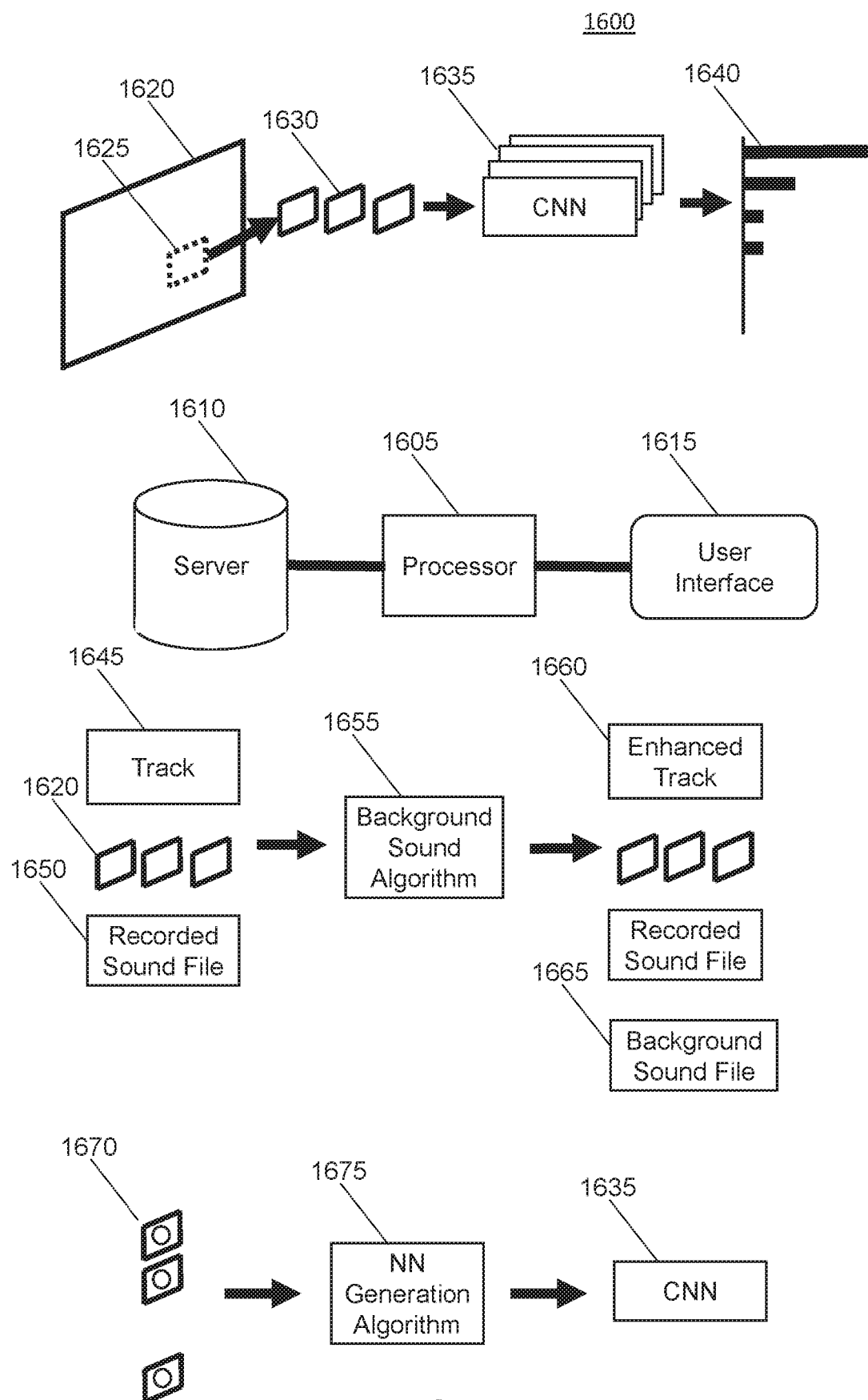


FIG. 16

17/18

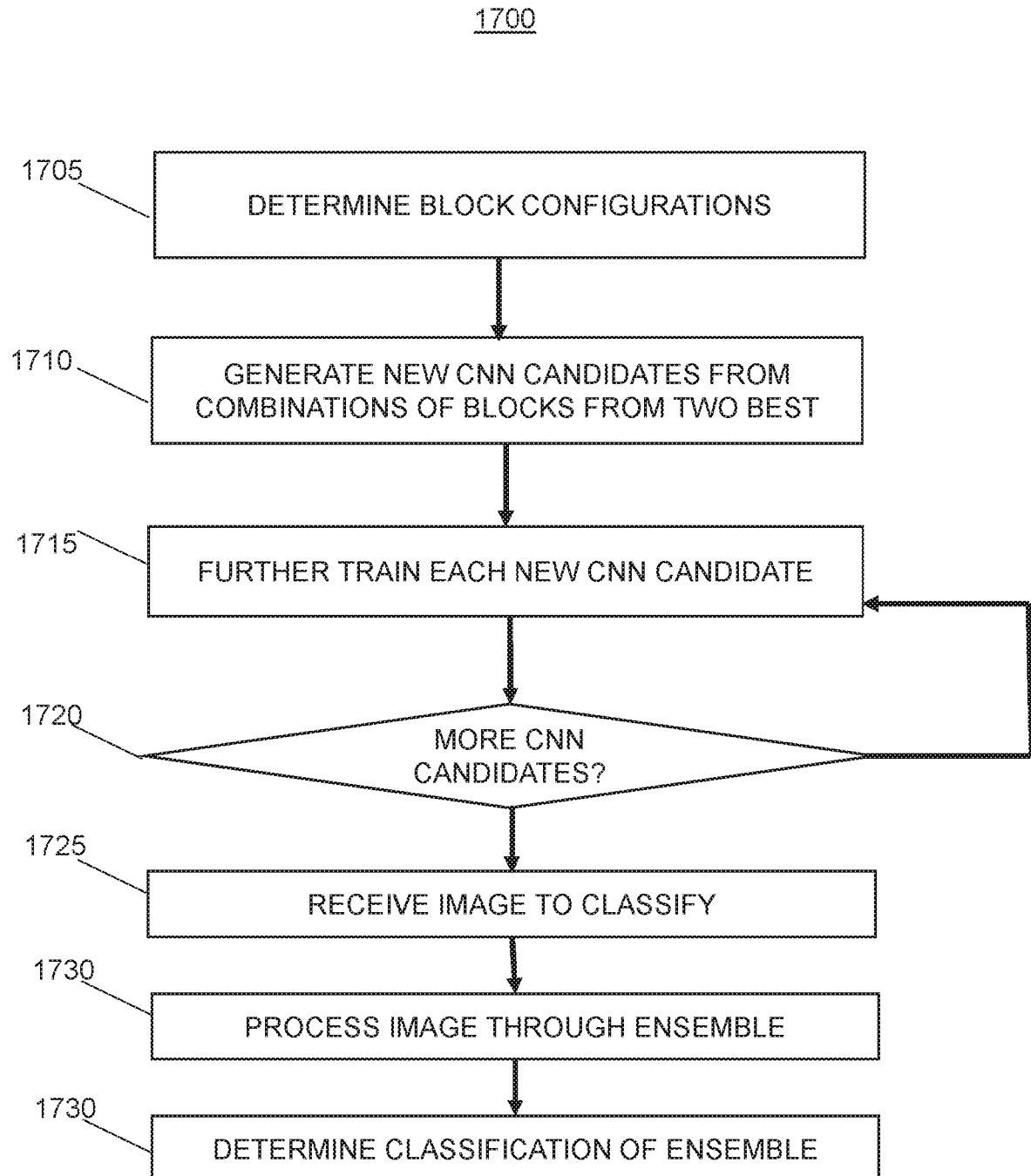


FIG. 17

18/18

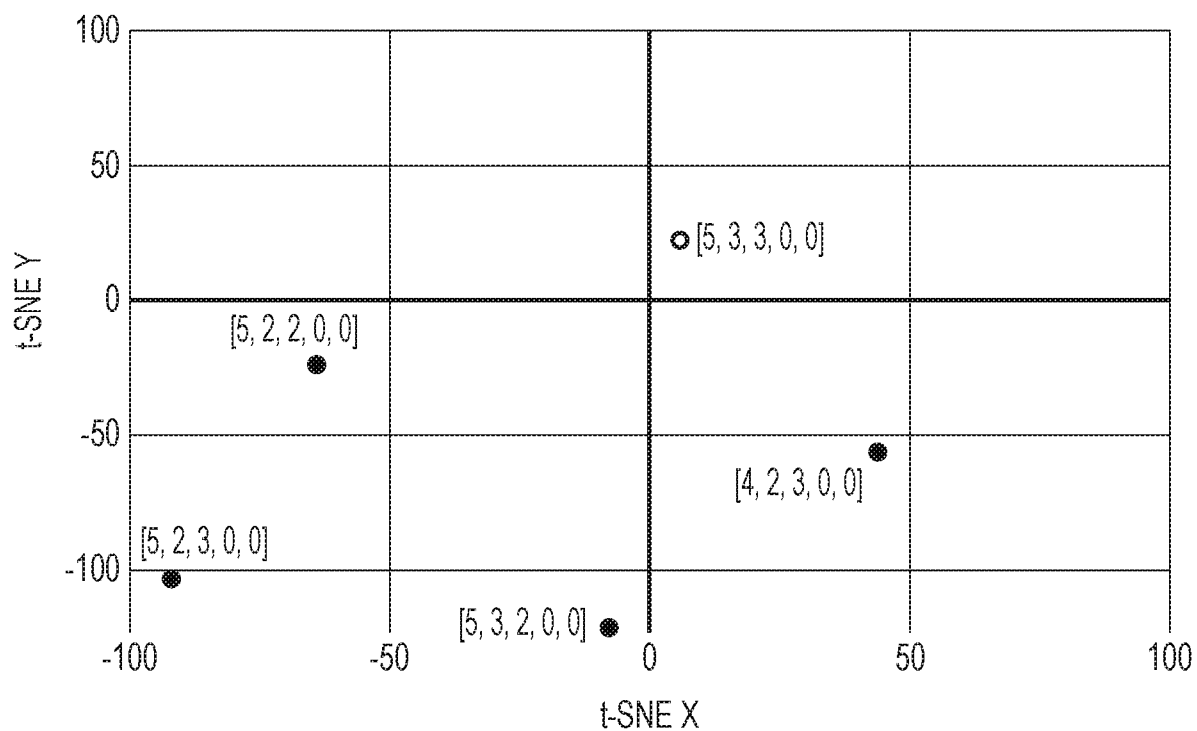


FIG. 18

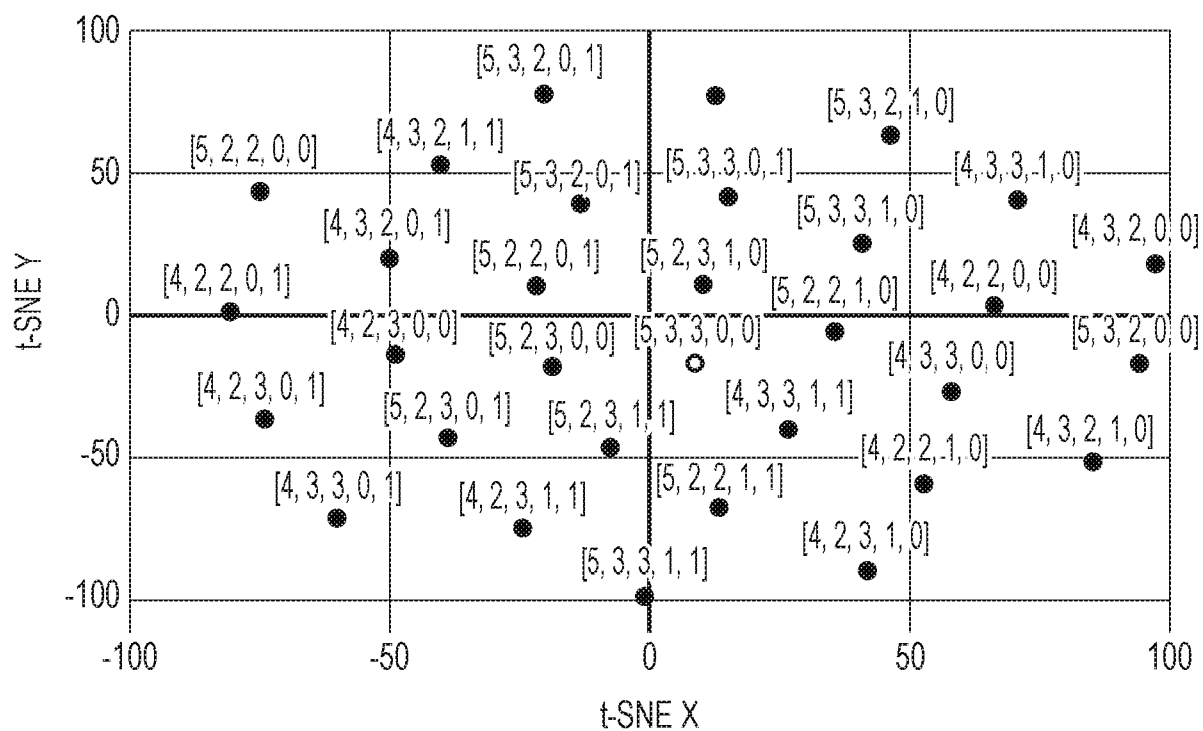


FIG. 19

INTERNATIONAL SEARCH REPORT

International application No
PCT/GB2019/053198

A. CLASSIFICATION OF SUBJECT MATTER INV. G06N3/00 G06N3/04 G06N3/08 G11B27/10 ADD.		
According to International Patent Classification (IPC) or to both national classification and IPC		
B. FIELDS SEARCHED Minimum documentation searched (classification system followed by classification symbols) G06N G11B		
Documentation searched other than minimum documentation to the extent that such documents are included in the fields searched		
Electronic data base consulted during the international search (name of data base and, where practicable, search terms used) EPO-Internal, WPI Data		
C. DOCUMENTS CONSIDERED TO BE RELEVANT		
Category*	Citation of document, with indication, where appropriate, of the relevant passages	Relevant to claim No.
X Y A	WO 2018/156942 A1 (GOOGLE LLC [US]) 30 August 2018 (2018-08-30) figures 1-3 paragraphs [0003] - [0005], [0010] - [0085] ----- EP 3 065 085 A1 (ACCENTURE GLOBAL SERVICES LTD [IE]) 7 September 2016 (2016-09-07) figures 1-6 paragraphs [0003] - [0045] ----- -/-	1-17, 19-21 18,22-24 1-24
☒ Further documents are listed in the continuation of Box C. ☒ See patent family annex.		
* Special categories of cited documents : "A" document defining the general state of the art which is not considered to be of particular relevance "E" earlier application or patent but published on or after the international filing date "L" document which may throw doubts on priority claim(s) or which is cited to establish the publication date of another citation or other special reason (as specified) "O" document referring to an oral disclosure, use, exhibition or other means "P" document published prior to the international filing date but later than the priority date claimed "T" later document published after the international filing date or priority date and not in conflict with the application but cited to understand the principle or theory underlying the invention "X" document of particular relevance; the claimed invention cannot be considered novel or cannot be considered to involve an inventive step when the document is taken alone "Y" document of particular relevance; the claimed invention cannot be considered to involve an inventive step when the document is combined with one or more other such documents, such combination being obvious to a person skilled in the art "&" document member of the same patent family		
Date of the actual completion of the international search	Date of mailing of the international search report	
13 February 2020	21/02/2020	
Name and mailing address of the ISA/ European Patent Office, P.B. 5818 Patentlaan 2 NL - 2280 HV Rijswijk Tel. (+31-70) 340-2040, Fax: (+31-70) 340-3016	Authorized officer Freitas, Arthur	

INTERNATIONAL SEARCH REPORT

International application No
PCT/GB2019/053198

C(Continuation). DOCUMENTS CONSIDERED TO BE RELEVANT		
Category*	Citation of document, with indication, where appropriate, of the relevant passages	Relevant to claim No.
Y	PEDRO MORGADO ET AL: "Self-Supervised Generation of Spatial Audio for 360 Video", ARXIV.ORG, CORNELL UNIVERSITY LIBRARY, 201 OLIN LIBRARY CORNELL UNIVERSITY ITHACA, NY 14853, 7 September 2018 (2018-09-07), XP081188579,	22-24
A	abstract; figures 1-7 page 1, paragraph 1 - page 8, paragraph 3 -----	1-21
A	DINGZEYU LI ET AL: "Scene-Aware Audio for 360textdegree Videos", ARXIV.ORG, CORNELL UNIVERSITY LIBRARY, 201 OLIN LIBRARY CORNELL UNIVERSITY ITHACA, NY 14853, 13 May 2018 (2018-05-13), XP080877585, DOI: 10.1145/3197517.3201391 abstract; figures 1-17 page 2, left-hand column, paragraph 1 - page 11, left-hand column, paragraph 4 -----	1-24
A	Anonymous: "Particle swarm optimization", 24 October 2018 (2018-10-24), XP055666066, Retrieved from the Internet: URL:https://en.wikipedia.org/w/index.php?title=Particle_swarm_optimization&oldid=865561206 [retrieved on 2020-02-06] page 1, paragraph 1 - page 6, paragraph 1 -----	1-24
Y	KYRIAKOS KENTZOGLANAKIS ET AL: "A Swarm Intelligence Framework for Reconstructing Gene Networks: Searching for Biologically Plausible Architectures", IEEE/ACM TRANSACTIONS ON COMPUTATIONAL BIOLOGY AND BIOINFORMATICS, IEEE SERVICE CENTER, NEW YORK, NY, US, vol. 9, no. 2, 31 March 2012 (2012-03-31), pages 358-371, XP011412142, ISSN: 1545-5963, DOI: 10.1109/TCBB.2011.87	18
A	abstract; figure 11 page 360, right-hand column, paragraph 4 - page 370, left-hand column, paragraph 1 -----	1-17, 19-24

INTERNATIONAL SEARCH REPORT

Information on patent family members

International application No

PCT/GB2019/053198

Patent document cited in search report	Publication date	Patent family member(s)	Publication date
WO 2018156942 A1	30-08-2018	CN 110366734 A EP 3574453 A1 KR 20190117713 A US 2019370659 A1 WO 2018156942 A1	22-10-2019 04-12-2019 16-10-2019 05-12-2019 30-08-2018
EP 3065085 A1	07-09-2016	AU 2016201292 A1 CA 2921672 A1 CN 105938559 A EP 3065085 A1 US 2016259994 A1	22-09-2016 04-09-2016 14-09-2016 07-09-2016 08-09-2016